

1 Highlights

2 **R4Tun: a reasoning-based multi-agent framework for segmental** 3 **tunnel analysis in point clouds**

4 Xinghui Tao, Guangming Wang *, Jelena Ninić, Brian Sheil

- 5 • Introduce R4Tun, a reasoning-driven multi-agent framework for tunnel
6 point clouds.
- 7 • Use LLM reasoning for context-aware parameter adaptation and trans-
8 parent decisions.
- 9 • One-step reflection drives emergent self-correction by reprocessing weak
10 blocks.
- 11 • Achieved 57% relative improvement compared to SOTA benchmarks.

R4Tun: a reasoning-based multi-agent framework for segmental tunnel analysis in point clouds

Xinghui Tao^a, Guangming Wang ^{*a}, Jelena Ninić^b, Brian Sheil^a

^a*Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK*

^b*Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, Durham, UK*

Abstract

Automated inspection of segmental tunnel linings is increasingly required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks. While modern laser scanning can capture large, geometrically rich point-cloud datasets, reliably segmenting these into individual components remains a challenging step for downstream deformation analysis and defect localisation. Current pipelines depend heavily on expert-driven hard-coded features, annotated datasets, or specifically tuned parameters, undermining their suitability for field deployment at scale. This paper presents **R4Tun**, a reasoning-based multi-agent framework that integrates an expert-designed segmentation pipeline with constrained large language model (LLM) driven reasoning layers for autonomous context-aware parameter adaptation. Evaluated on the open-source Seg2Tunnel benchmark under controlled deviation from an expert-tuned reference, R4Tun improves average mIoU from 28.5% to 44.8% (+57% relative), and in off-reference cases prevents baseline collapse from average mIoU 3.8% to 34.9% (+818% relative). The results show that R4Tun exhibits greater robustness by adapting parameters explicitly, as tunnels drift from the expert-tuned reference conditions. The proposed reasoning-based framework demonstrates the potential of multi-agent AI systems to act as transparent, engineer-guided systems for scalable automated tunnel lining inspection. Our code and data are available at <https://github.com/Tao-Robominds/R4Tun>.

Keywords: R4Tun, Tunnel inspection, Point cloud segmentation, LLMs, Multi-agent systems, Reasoning

1. Introduction

Automated inspection of segmental tunnel linings is increasingly required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks (Attard et al., 2018). Unlike 2D image data, which collapses geometry and is highly sensitive to viewpoint, lighting, and surface appearance, 3D spatial data preserves true geometric relationships and scale, making it inherently better suited to characterising tunnel lining shape, alignment, and deformation in complex underground environments. Reliable segmentation of structural components from the 3D data is a prerequisite for downstream tasks such as deformation assessment and defect localisation (Weidner and Walton, 2024). Although modern laser-scanning technologies, now increasingly deployed as *mobile laser scanning (MLS)* systems, enable high-fidelity tunnel capture, the resulting measurements typically contain mixed structural elements, occlusions, and noise. This makes direct extraction of lining components challenging (Huang et al., 2021; Sjölander et al., 2023). In practice, engineers must also operate across projects where tunnel geometry, lining condition, and data acquisition conditions (e.g., viewpoint coverage, motion, and resolution in MLS surveys) vary substantially. As a result, expert oversight remains necessary to establish trust in automation (Montero et al., 2015; Strauss et al., 2020; Camuffo et al., 2022).

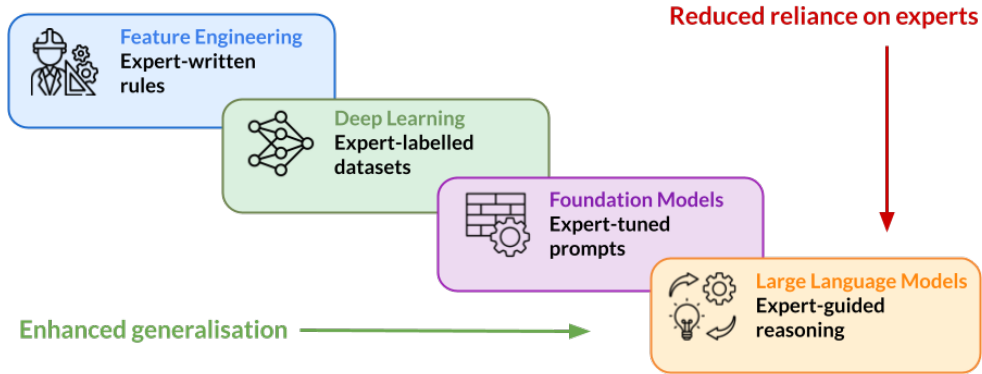


Figure 1: Evolution of automated segmentation paradigms toward enhanced generalisation. R4Tun advances beyond the expert-tuning approach by incorporating expert-guided reasoning.

Three main approaches have emerged for tunnel point-cloud segmentation

(Figure 1). The first is feature-engineering, which encodes domain knowledge into deterministic rules, typically defined through geometric thresholds, line/edge features, curvature descriptors, or clustering criteria (Duda and Hart, 1972; Fischler and Bolles, 1981; Ester et al., 1996; Pauly et al., 2002). Because these rules are explicit and interpretable, engineers can readily audit and refine them, and such pipelines therefore remain widely used in practice. However, they are inherently brittle: rules tuned to a specific tunnel geometry or scanning configuration often fail under changed conditions, requiring extensive manual reconfiguration (Weidner and Walton, 2024). The second approach is supervised deep learning. By learning hierarchical representations from large annotated datasets, deep learning models reduce reliance on hand-crafted geometric rules (Qi et al., 2017; Hu et al., 2020; Schult et al., 2023; Kolodiazhnyi et al., 2024; Cha et al., 2024; Su et al., 2024). However, such models require large labelled datasets, significant computational resources, and periodic retraining. Furthermore, their internal decision logic is opaque to interpret, which further reinforces engineers’ concerns about the trustworthiness of automated pipelines. The third approach, involving the use of foundation models (Kirillov et al., 2023; Caron et al., 2021), has recently been explored to mitigate some of these limitations. These models are pre-trained on large and diverse datasets to achieve zero-shot generalisation guided by user prompts, reducing the need for task-specific labelled data and retraining (Bommasani et al., 2021; Franceschelli et al., 2025). Prompting here refers to supplying structured cues that steer the foundation segmentation model toward the target structure. In civil engineering, such models have been applied to tunnel component segmentation, structural health monitoring, and Scan-to-BIM workflows (R et al., 2024; Wang et al., 2024; Pan et al., 2023). Among them, SAM4Tun (Ye et al., 2025), built on Meta’s Segment Anything Model (SAM) (Kirillov et al., 2023), introduces a meaningful step forward for tunnel lining segmentation by combining geometric preprocessing, image-based line detection, and prompt-based segmentation into a unified pipeline. SAM4Tun has been shown to achieve strong performance under expert parameter tuning, yet remains highly sensitive to preprocessing and prompting choices. Deviations in tunnel geometry or scanning conditions from those assumed during tuning can lead to sharp performance degradation. For practitioners, this sensitivity limits robustness and reproducibility beyond the narrow conditions for which the method is calibrated.

Most recently, advances in natural language processing have enabled large language models (LLMs) to become embedded in engineering workflows as

general-purpose assistants (Bradley et al., 2023; Ghafarollahi and Buehler, 2024; Qian et al., 2024; Hong et al., 2024; Chen et al., 2024; Garcia et al., 2024). Conventional LLMs trained purely for next-token prediction, such as GPT-3.5 or DeepSeek-V3 (OpenAI, 2024; DeepSeek-AI, 2025a) are often perceived as prone to hallucination and unreliable multi-step reasoning (Bender et al., 2021; Ouyang et al., 2022; Farquhar et al., 2024). To address this limitation, reasoning-enabled LLMs incorporate post-training on curated, step-by-step reasoning traces, enabling them to articulate intermediate steps explicitly and improving logical consistency, interpretability, and verifiability. (Wei et al., 2023; Kojima et al., 2023; Xiang et al., 2025). Newer models, such as OpenAI’s O-series and DeepSeek’s R1 (OpenAI, 2025; DeepSeek-AI, 2025b), demonstrate emergent capabilities including self-reflection and error correction (Ferraz et al., 2024). However, for engineers, a key challenge is the integration of such reasoning models in a way that enhances the generalisation of expert-designed pipelines while reducing the trust issues associated with black-box deep learning, including foundation models.

In this paper, we introduce R4Tun, a reasoning-based multi-agent framework that augments the engineer-designed SAM4Tun pipeline with reasoning-enabled LLMs acting as white-box controllers. R4Tun preserves SAM4Tun’s geometric operations and sequential workflow, while replacing static, expert-tuned parameters with adaptive agents guided by constrained, context-sensitive reasoning. Each agent operates within explicit, engineer-specified bounds: it receives a compact description of the current tunnel state, a memory containing an expert-tuned reference, and a human-readable knowledge base, and then produces structured parameter updates via Chain-of-Thought (CoT) reasoning (Wei et al., 2023; Xiang et al., 2025; Zhang et al., 2023). CoT refers to a reasoning process in which the model generates explicit intermediate steps instead of jumping directly to a final answer. In R4Tun, stage-level CoTs enable agents to adapt parameter changes by reasoning over its specific geometric and data-quality characteristics. Subsequently, a reflective CoT analyses the final segmentation, identifies underperforming regions, and proposes targeted corrections (Shinn et al., 2023; Yao et al., 2023; Zhang et al., 2024; Liu et al., 2025; Gao et al., 2025). All intermediate reasoning steps and parameter updates are logged, allowing engineers to inspect, modify, or override the system’s behaviour at any time.

The key contributions of this work are as follows:

- (i) We introduce a new reasoning-based multi-agent framework that integrates engineer-designed prompt-based segmentation pipelines built on foun-

116 dation models; the framework preserves the original workflow whilst dele-
117 gating parameter adaptation to reasoning agents. Although evaluated on
118 tunnel linings, it is pipeline-agnostic and readily extends to other geometry-
119 and data-quality-sensitive assets (e.g., pipes, shafts, bridges, retaining walls,
120 roads, and buildings).

121 (ii) We design a reasoning layer in which agents operate within con-
122 strained parameter spaces, access human-editable memory and knowledge,
123 and produce logged CoT traces so that every adaptation step can be in-
124 spected, adjusted, or overridden.

125 (iii) We show that this design improves stability and failure avoidance
126 across varying tunnel conditions, reduces manual retuning effort, and gives
127 rise to emergent diagnostic behaviour that provides a foundation for continual
128 self-improvement over time.

129 The rest of this paper is organised as follows. Section 2 presents the
130 methodology, describing the R4Tun architecture, the datasets, and the ex-
131 perimental setup. Section 3 reports the results and discusses system perfor-
132 mance. Section 4 summarises key limitations, and Section 5 concludes the
133 study, and outlines future research directions.

134 2. Methodology

135 The R4Tun workflow (Figure 2) augments an engineer-designed prompt-
136 based segmentation pipeline SAM4Tun with a structured reasoning layer that
137 links its underlying LLMs, evolving context, CoT-based decision-making, and
138 reflection. The framework preserves SAM4Tun’s structure, in which the 3D
139 tunnel point cloud is (1) unfolded into a panoramic depth map, (2) denoised,
140 and (3) enhanced; (4) ring boundaries are then detected and used to construct
141 template-based prompts for structural segments, and (5) the resulting seg-
142 mentation is reprojected into 3D labels. SAM4Tun uses engineer-designed
143 geometric and image-processing steps (e.g., line/edge detection and region
144 proposals) to generate prompts (e.g. structured inputs such as points, boxes,
145 or coarse masks) that indicate where lining segments are expected. Given
146 these prompts, SAM produces candidate masks that are then filtered and
147 merged back into the tunnel domain. SAM4Tun’s performance depends on
148 a set of pipeline parameters that control preprocessing, prompt generation,
149 and post-processing (e.g., thresholds, window sizes, sampling densities, and
150 merging criteria). These parameters must often be retuned when tunnel ge-
151 ometry or point-cloud quality changes, motivating the adaptive reasoning

152 layer introduced in R4Tun. To reduce possible hallucination, R4Tun (i) pre-
 153 serves all geometric and image-processing algorithms from SAM4Tun, (ii)
 154 restricts each agent to adjusting parameters within predefined bounds, and
 155 (iii) logs every reasoning trace and applied configuration so that decisions can
 156 be inspected and overridden. Each key processing stage (unfolding, denois-
 157 ing, enhancing, and prompting) is managed by an individual agent whose
 158 LLM uses CoT reasoning over the evolving point cloud characteristics to
 159 propose appropriate parameter. After the initial segmentation, a reflective
 160 agent reviews the output, identifies regions with poor coverage, and applies
 161 targeted refinement.

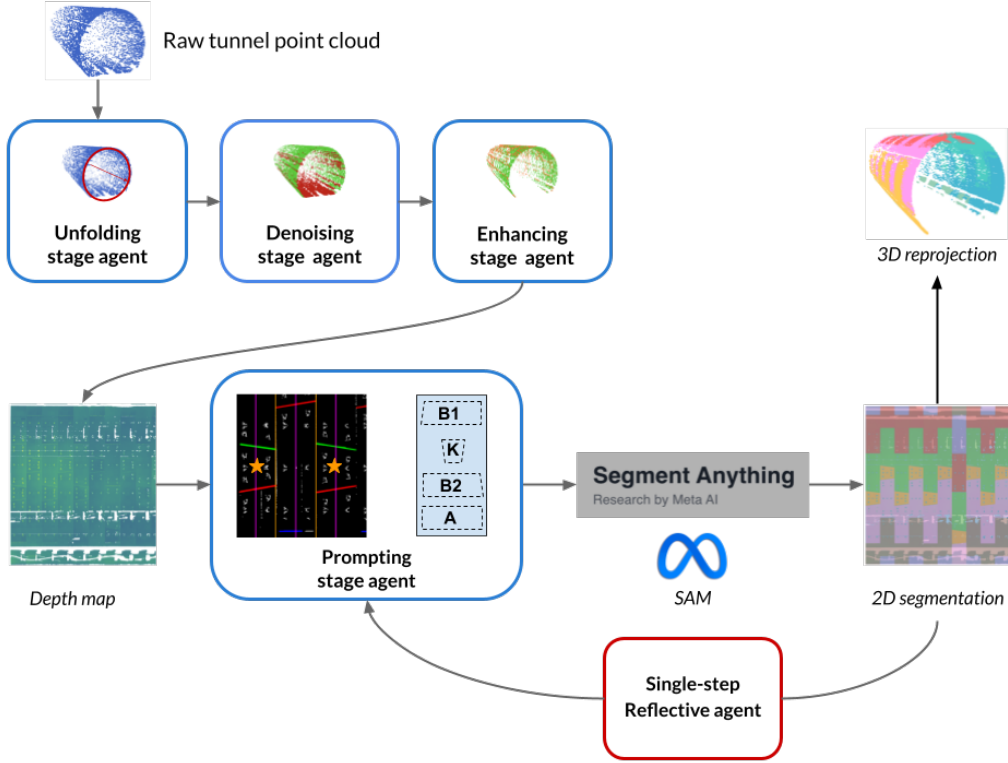


Figure 2: The R4Tun multi-agent architecture for tunnel segmentation. Four stage agents (unfolding, denoising, enhancement, and prompting) adapt their parameters through CoT reasoning to generate an initial segmentation. A separate reflective agent then evaluates the output and performs a targeted refinement.

162 Each R4Tun agent is a self-contained unit that integrates perception,
 163 reasoning, and action within a shared context (Figure 3). The context com-

164 bins three elements (memory, knowledge, and state), which together inform
 165 the agent’s processing decisions. Within this context, an analyst function
 166 performs structured CoT reasoning to interpret the current conditions and
 167 propose parameter adjustments, while a coder function executes the corre-
 168 sponding operations. While both the stage agents and the reflective agent
 169 follow a shared CoT reasoning structure, they differ in purpose: stage agents
 170 adapt parameters within their specific processing stages, whereas the reflec-
 171 tive agent intervenes after segmentation to analyse weak regions and apply
 172 a targeted refinement, thereby improving the final mIoU (and per-class IoU)
 173 in evaluation. This architecture turns a static-parameter pipeline into a con-
 174 trolled optimisation loop: every change is linked to an explicit reasoning
 175 trace, enabling engineers to trace, interpret, and override parameter updates
 176 whenever needed.

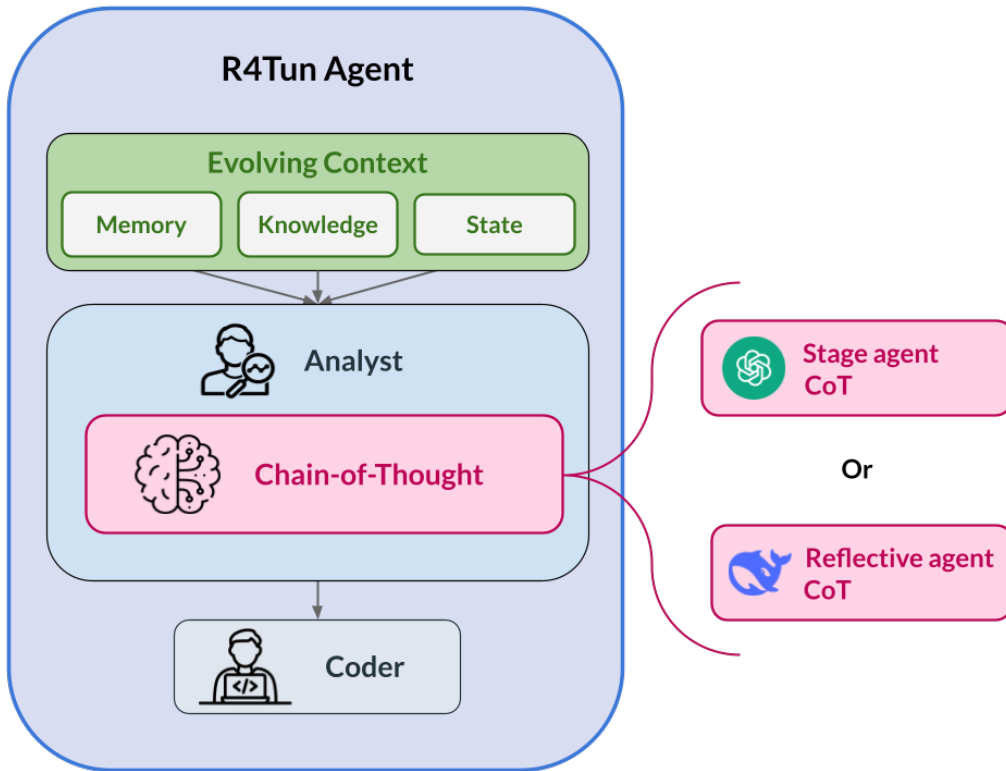


Figure 3: R4Tun agent architecture. Each agent maintains an evolving context (memory, knowledge, state) that informs an analyst component performing CoT reasoning. A coder component executes the reasoning outputs.

2.1. CoT Reasoning

Reasoning-enabled LLMs form the cognitive core of each agent within R4Tun. In this work we employ two complementary models. For the stage agents, we use GPT-O4, whose coding and numerical capabilities make it well suited to generating and adjusting the scalar parameters at each processing stage. For the reflective agent, we adopt DeepSeek R1, selected for its stronger diagnostic and self-reflective behaviour and its ability to articulate detailed analogy-based reasoning. Both models are reasoning-enabled LLMs released with the capability to generate step-by-step rationales, producing multi-step logical sequences rather than single-shot answers.

CoT reasoning provides the analytical structure for R4Tun’s decision-making: agents diagnose how the current tunnel’s geometry and point-cloud quality differ from sample conditions, infer which pipeline assumptions are being violated, and then select justified, stage-specific parameter updates within a constrained search space, recording the rationale so each intervention is interpretable and overridable. It enables each agent to generate transparent, step-by-step reasoning traces during parameter adaptation, transforming otherwise implicit model behaviour into explicit logical sequences. In R4Tun, two complementary CoT structures following a shared schemes are employed: a stage agent CoT that governs operations through the pipeline, and a reflective agent CoT that performs post-segmentation evaluation.

The stage agent CoT (Figure 4) begins with anchoring, in which the agent compares the current tunnel’s geometric and processing characteristics (e.g., diameter, point density, and noise level) to a stored reference configuration for the same stage. Here, *stage parameters* refer to the fixed, engineer-designed numeric settings that govern the underlying operation (such as thresholds, sampling densities, window sizes, or iteration limits). When the tunnel is strongly off-reference, the agent treats the case as *weakly anchored* and defaults to conservative, bounded adjustments guided primarily by measured state, rather than broad analogy-based corrections; such cases also motivate retrieving additional, more diverse references to improve anchoring in the future.

The agent then performs a diagnostic inspection to attribute the deviation to a plausible cause and to identify which stage parameters are implicated. For example, after observing that the new tunnel point cloud is *denser* and has a *slightly larger diameter* than the reference (Figure 4), the agent infers that the ellipse-fitting step may become less stable and recommends lowering the corresponding acceptance threshold (e.g., “Use a lower

215 `ellipse_fit_threshold` to maintain stability”). Based on this assessment,
216 parameter adaptation proposes concrete updates within predefined bounds
217 and documents the rationale using stored tuning guidelines, such as “Update
218 `ellipse_fit_threshold` to 0.8.”

219 A brief validation step then performs a lightweight logical and consistency
220 check (e.g., confirming that a lower `ellipse_fit_threshold` is expected to
221 improve robustness under denser data) rather than acting as a full perfor-
222 mance evaluation. Finally, the selected parameters and their reasoning trace
223 are packaged into a structured JSON object (e.g., `{"ellipse_fit_threshold":`
224 `0.8, "ellipse_fit_probability": 0.95}`), which is passed to the coder
225 function to execute the underlying algorithms for that stage.

226 *The reflective agent CoT* (Figure 5) applies the same reasoning structure
227 to post-segmentation refinement, but is invoked only as a single corrective
228 pass. Rather than anchoring to a reference, it begins with targeting: it scans
229 the segmented tunnel for weak coverage using simple state signals (e.g., point
230 density aggregated over segment blocks) and, in our experiments, selects only
231 the worst-covered region for intervention. For instance, the agent may flag
232 that “A2-block is the weakest with only 21,413 points (below the threshold
233 of 81,564)”.

234 The subsequent diagnostic inspection reasons about plausible causes of
235 the weakness (e.g., concluding that overall density is high and the issue is
236 likely related to assignment), and parameter adaptation proposes a targeted
237 correction that affects only the implicated control variables (e.g., adjusting
238 the segment-processing order so the weak region is handled first). Valida-
239 tion then performs a lightweight consistency check to ensure the change is
240 logically coherent and unlikely to degrade other regions, before the updated
241 JSON parameters (e.g., `segment_order` and `segment_per_ring`) are passed
242 to the coder for execution. Importantly, Figure 5 visualises only a representa-
243 tive excerpt of the reflective agent’s reasoning and resulting parameter edits
244 to highlight the most salient emergent behaviour. We provide additional
245 examples and extended logs of such behaviours in Appendix Appendix A.

246 2.2. *Evolving Context*

247 Context engineering refers to the systematic design of the information
248 provided to a reasoning model during CoT processing (Xu et al., 2024; Mei
249 et al., 2025). Recent work has shown how the organisation and relevance of
250 the provided context can strongly influence model performance (Anthropic,

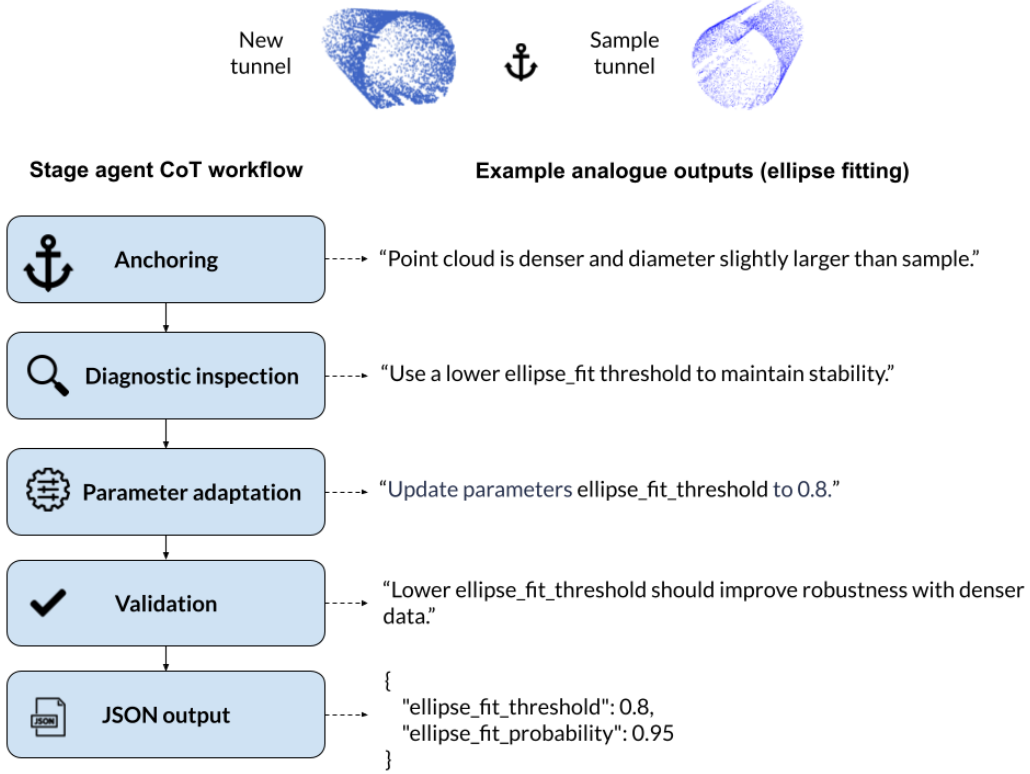


Figure 4: The left column shows the five CoT phases: (1) Anchoring, (2) Diagnostic inspection, (3) Parameter adaptation, (4) Validation, and (5) JSON output. The right column provides a short CoT output example for each phase, demonstrating how the agent interprets tunnel characteristics, adjusts parameters, validates updates, and exports structured outputs. Dashed arrows link each step to its corresponding example.

251 2025; OpenAI, 2025) . Within R4Tun, three interacting components (mem-
 252 ory, knowledge and state) form an evolving context that grounds the agent’s
 253 reasoning in structured and interpretable information.

254 *Memory* (Figure 6) provides the reference baseline for R4Tun’s reason-
 255 ing. In the current implementation, it stores a single expert-tuned reference
 256 configuration from the open-source SAM4Tun pipeline, together with a com-
 257 pact summary of the corresponding tunnel characteristics. Agents use this
 258 reference for comparison during the anchoring phase, interpreting new inputs
 259 relative to known operating categories rather than assuming universal valid-
 260 ity. When the current tunnel deviates substantially from this reference, the

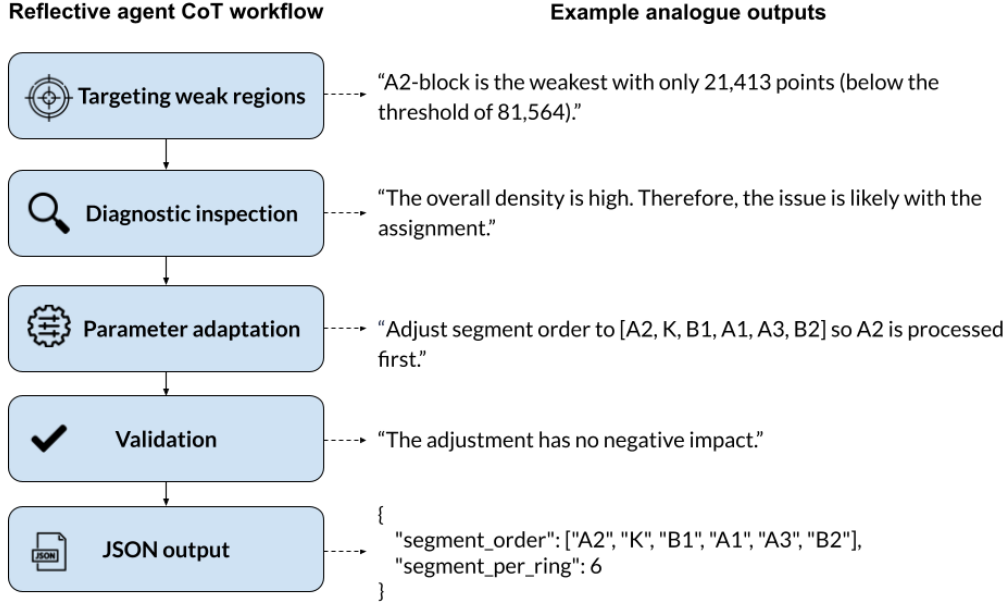


Figure 5: Reflective agent CoT reasoning workflow for post-segmentation correction. The left column shows the five CoT phases: (1) Targeting weak regions, (2) Diagnostic inspection, (3) Parameter adaptation, (4) Validation, and (5) JSON output. The right column provides one short example CoT output for each phase, showing how the agent identifies the weakest segment, infers likely causes, proposes a targeted adjustment, checks its logical consistency, and exports the revised parameters. Dashed arrows link each step to its corresponding example.

261 anchoring is treated as weak and the agent prioritises conservative, bounded
 262 updates driven primarily by the measured state.

263 *Knowledge* (Figure 7) supplies domain-specific guidance in the form of
 264 concise parameter definitions and tuning rules. During reasoning, the rele-
 265 vant subset of this knowledge is injected into the stage agent’s context, so that
 266 parameter updates are grounded in known heuristics and remain appropriate
 267 for the current task. Because this knowledge is human-editable, engineers can
 268 revise, extend, or tighten the rules as they refine their understanding of how
 269 the pipeline should behave. In this way, knowledge constrains the agent’s
 270 possible actions, preventing arbitrary behaviour and keeping parameter up-
 271 dates within their validated ranges.

272 *State* (Figure 8) captures the evolving geometric and statistical properties
 273 of the point cloud across processing stages. By abstracting raw data into

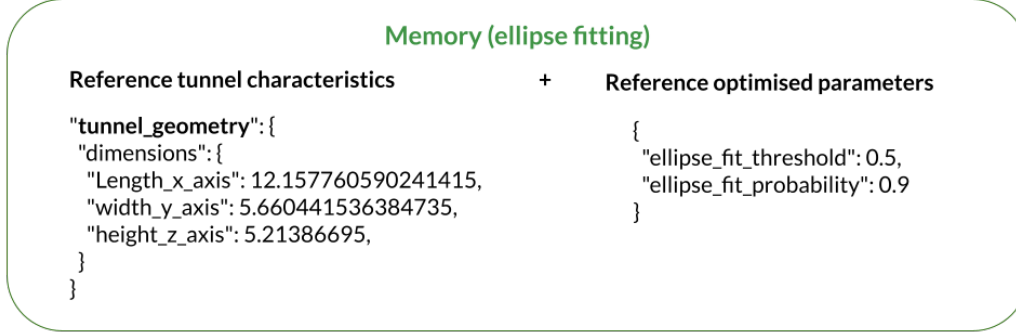


Figure 6: Memory stores reference tunnel characteristics *and* optimised parameter values provided by domain experts (illustrated here using the ellipse-fitting step), providing a reference baseline for CoT reasoning.

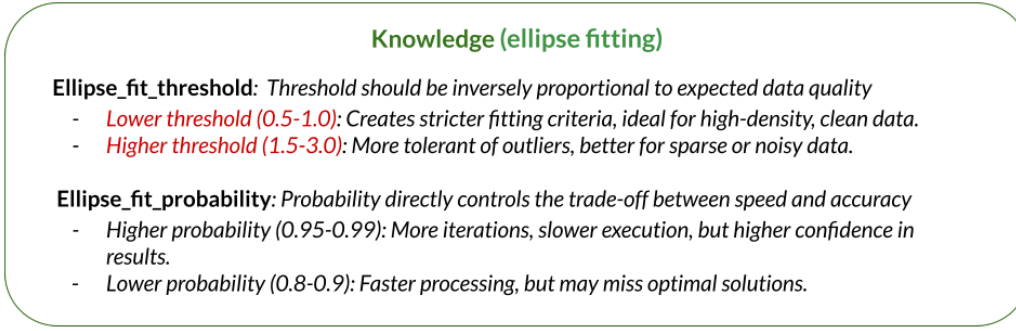


Figure 7: Knowledge contains parameter definition and tuning guidelines used to adjust processing values (illustrated here using the ellipse fitting parameters).

274 structured cues (e.g., local curvature, diameter change, and point density),
 275 state provides agents with a stage-relevant summary of current conditions.
 276 This enables reasoning over meaningful spatial patterns while reducing the
 277 cognitive load associated with full high-dimensional point cloud inputs. At
 278 each stage, the agent reads the current state, compares it to memory, consults
 279 knowledge, and then reasons toward an informed and grounded update.

280 2.3. Agents in Operation

281 For each processing stage $s \in \mathcal{S}$, let $\mathbf{p}_s \in \mathbb{R}^{d_s}$ denote the vector of d_s
 282 tunable, engineer-defined parameters for that stage (e.g., thresholds), and
 283 let $\Delta\mathbf{p}_s \in \mathbb{R}^{d_s}$ denote the stage agent’s proposed parameter increment. The
 284 agent conditions on the current context $\mathcal{C}_t = (\mathcal{M}, \mathcal{K}, \mathbf{x}_t)$, where $\mathcal{M}$ is the

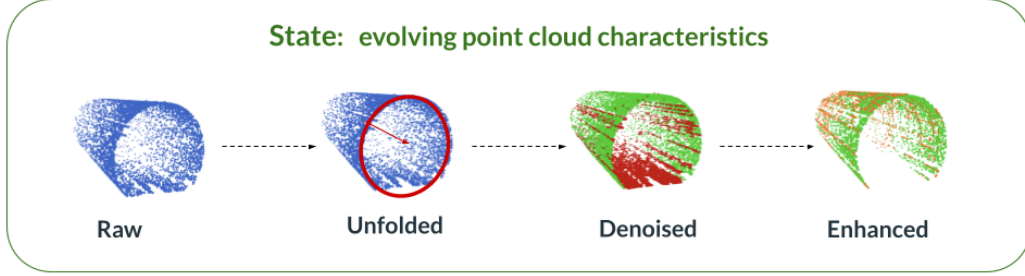


Figure 8: State captures evolving geometric properties of the point cloud across stages (Raw → Unfolded → Denoised → Enhanced), enabling agents to reason over changing spatial patterns.

285 editable memory (reference configurations and prior outcomes), $\mathcal{K}$ is the stage
 286 knowledge base (tuning guidelines and constraints), and $\mathbf{x}_t$ are measured
 287 state features summarising the current tunnel and intermediate outputs (e.g.,
 288 density, diameter, and quality statistics). The stage agent implements a
 289 bounded update,

$$\Delta \mathbf{p}_s = f_s(\mathcal{C}_t), \quad \mathbf{p}_s^{\text{new}} = \Pi_{[\mathbf{p}_s^{\min}, \mathbf{p}_s^{\max}]}(\mathbf{p}_s^{\text{old}} + \Delta \mathbf{p}_s), \quad (1)$$

290 where $f_s(\cdot)$ is the LLM-based reasoning function for stage s , $\mathbf{p}_s^{\min}$ and $\mathbf{p}_s^{\max}$ are
 291 engineer-specified per-parameter bounds, and $\Pi_{[\mathbf{p}_s^{\min}, \mathbf{p}_s^{\max}]}$ denotes element-
 292 wise projection (clipping) into this feasible interval. After segmentation, a
 293 dedicated Reflective Agent performs a single-step corrective pass by identi-
 294 fying the weakest segment block (under a coverage statistic computed from
 295 the predicted labels) and applying a targeted refinement. The full tunable
 296 parameter sets (names and defaults) for all stages are listed in Appendix Ap-
 297 pendix B, and additional reflective logs are reported in Appendix Appendix A.

298 *The unfolding agent* (Figure 9) establishes a cylindrical coordinate system
 299 for the tunnel to ensure consistent spatial analysis in downstream stages. It
 300 estimates a smooth centreline using a RANSAC-based procedure that iter-
 301 atively fits ellipses to cross-sections and selects the model with the highest
 302 inlier support (Fischler and Bolles, 1981). The resulting centreline defines the
 303 cylindrical frame used to project the 3D tunnel surface into a 2D panoramic
 304 image, where each pixel encodes radial distance to the centreline. This trans-
 305 formation regularises the spatial domain and produces a stable depth repre-
 306 sentation for subsequent filtering and segmentation. In this stage, the agent
 307 can tune a set of engineer-defined parameters that control slice construction
 308 and centreline fitting, including the slice half-thickness, longitudinal slice

spacing, RANSAC settings, centreline smoothing degree, and sampling density.

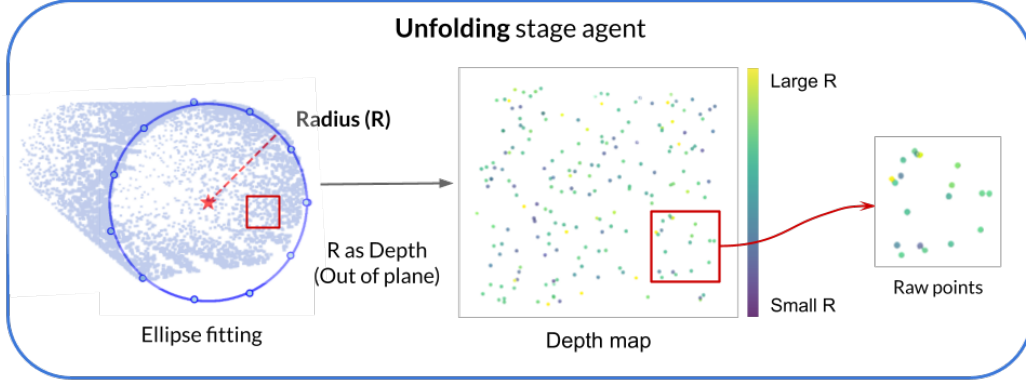


Figure 9: Stage-unfolding agent in operation. The left panel shows the fitted tunnel cross-section used to determine the reference center and radius for cylindrical unfolding. The middle panel presents the corresponding 2D panoramic depth map derived from the 3D point cloud. The right panel shows a zoomed region of the unfolded map.

The *denoising agent* (Figure 10) removes non-structural artefacts such as rails, cables, and scattered points that may interfere with segmentation. The underlying algorithm uses a grid-based radial-density filtering method inspired by the DBSCAN clustering principle (Ester et al., 1996), grouping points according to their local density in cylindrical coordinates. Dense, connected regions correspond to tunnel surfaces, while isolated points are rejected as noise. A pair of radial masks constrains filtering to the expected tunnel radius, preserving joint boundaries and ring edges while discarding outliers. This process produces a clean and continuous surface representation suitable for curvature analysis and interpolation in the next stage. In this stage, the agent can tune a set of engineer-defined parameters that control radial masking and density-based filtering, including the radial mask bounds (R_{low} , R_{high}), grid resolution, neighbourhood radius, minimum-density thresholds, and outlier-rejection criteria.

The *enhancing agent* (Figure 11) improves geometric continuity and surface completeness before projection into the 2D panoramic depth map. Local curvature analysis using Principal Component Analysis (PCA) on neighbourhood points is performed to guide point insertion. Points are inserted between neighbours when the curvature difference is small, indicating smooth surface regions, while regions with high curvature variation (boundaries/joints) are

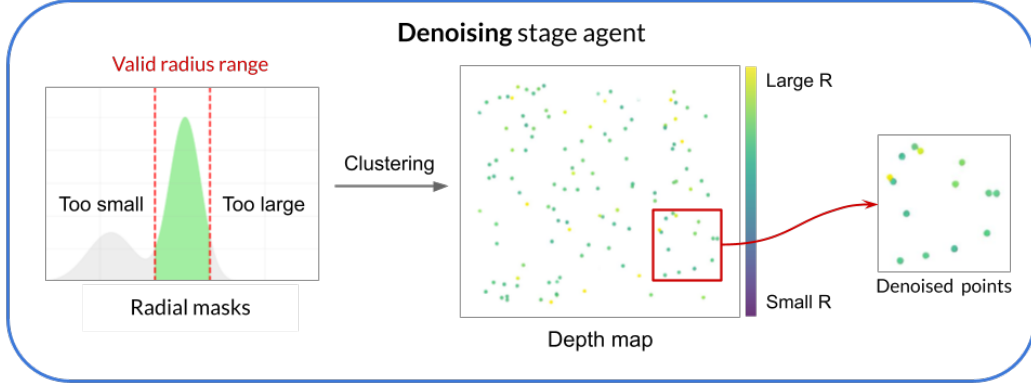


Figure 10: Denoising stage agent in operation. The left panel shows the radial-distance distribution used to filter out points that fall outside the expected tunnel lining surface. The middle panel displays the corresponding unfolded 2D depth map after filtering, highlighting the cleaned point distribution. The right panel presents a zoomed region of the denoised map.

331 preserved. A three-stage progressive upsampling approach inserts additional
 332 points at progressively finer resolutions to maintain surface continuity. Pixel
 333 interpolation enhances outlier points at joint locations. The refined surface is
 334 then projected into a panoramic depth map through pixel-level interpolation,
 335 producing an image that balances smooth panel regions with sharply defined
 336 joint boundaries. In this stage, the agent can tune a set of engineer-defined
 337 parameters that control curvature-guided insertion and multi-resolution up-
 338 sampling, including neighbourhood size for PCA curvature estimation, curva-
 339 ture similarity thresholds for insertion, stage-wise target spacing/resolution,
 340 insertion density per stage, and pixel interpolation settings around joint re-
 341 gions.

342 *The prompting agent* (Figure 12a) identifies joint boundaries and pre-
 343 pares the panoramic depth map for segmentation. The underlying algorithm
 344 applies a Hough-transform-based detector (Duda and Hart, 1972) to extract
 345 linear features corresponding to ring joints. These boundaries are then used
 346 to construct template-based prompts (e.g., point/region templates for key,
 347 base, and adjacent segments), which are passed to SAM to generate 2D masks
 348 that are subsequently reprojected into 3D labels. In this stage, the agent can
 349 tune a set of engineer-defined parameters spanning both (i) *boundary detec-*
 350 *tion* and (ii) *SAM prompting*, including binary and morphological settings,
 351 Hough thresholds and line-merging criteria, prompt density and placement

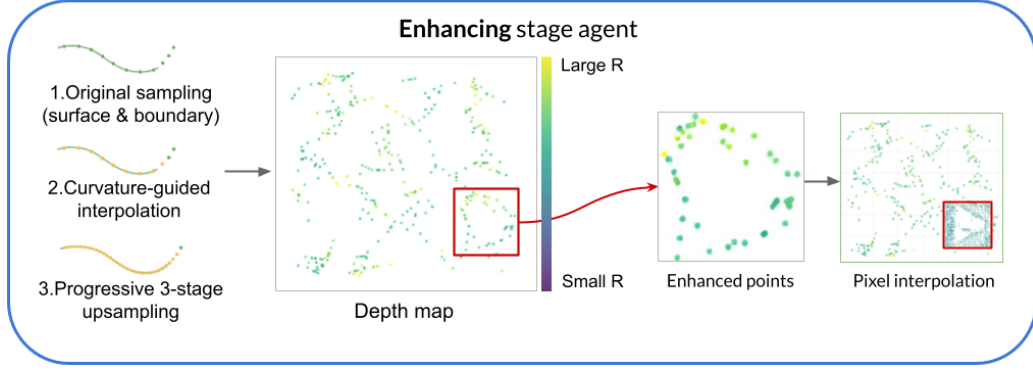


Figure 11: Enhancing stage agent in operation. The left panel illustrates curvature-guided surface and boundary interpolation along the tunnel lining, showing original sampling, curvature-guided interpolation, and progressive three-stage upsampling. The middle panel shows the corresponding unfolded 2D depth map after enhancement, where inserted points improve geometric continuity; the red box highlights a local region of interest. The right panels zoom into this region, first displaying the added points and then the pixel-level interpolation.

352 templates, segment ordering and grouping, and mask filtering/merging set-
 353 tings.

354 Following segmentation, the reflective agent (Figure 12b) is invoked once
 355 to inspect the predicted labels. It analyses the distribution of predicted
 356 labelled points across segment blocks (e.g., point counts or area coverage per
 357 block), identifies the least-covered region, and reuses the prompting context
 358 to run the reflective CoT. From this, the CoT diagnoses likely causes of under-
 359 segmentation and proposes a single, targeted parameter adjustment focused
 360 on the weakest segment block. The update is then applied in one corrective
 361 pass, and both the reasoning trace and final configuration are recorded for
 362 later inspection or override.

363 3. Datasets and Experiment

364 The experiments use Seg2Tunnel, a publicly available point cloud dataset
 365 comprising five real tunnels with variations in assembly type (staggered vs.
 366 continuous), diameter (5.5–7.5 m), ring length (1.2–1.8 m), and segment
 367 count (6–7). To keep the evaluation computationally practical on a single
 368 NVIDIA RTX 4090 GPU while preserving realistic tunnel-scale structure, we
 369 follow a partial-scene mode in which one contiguous subset is selected per

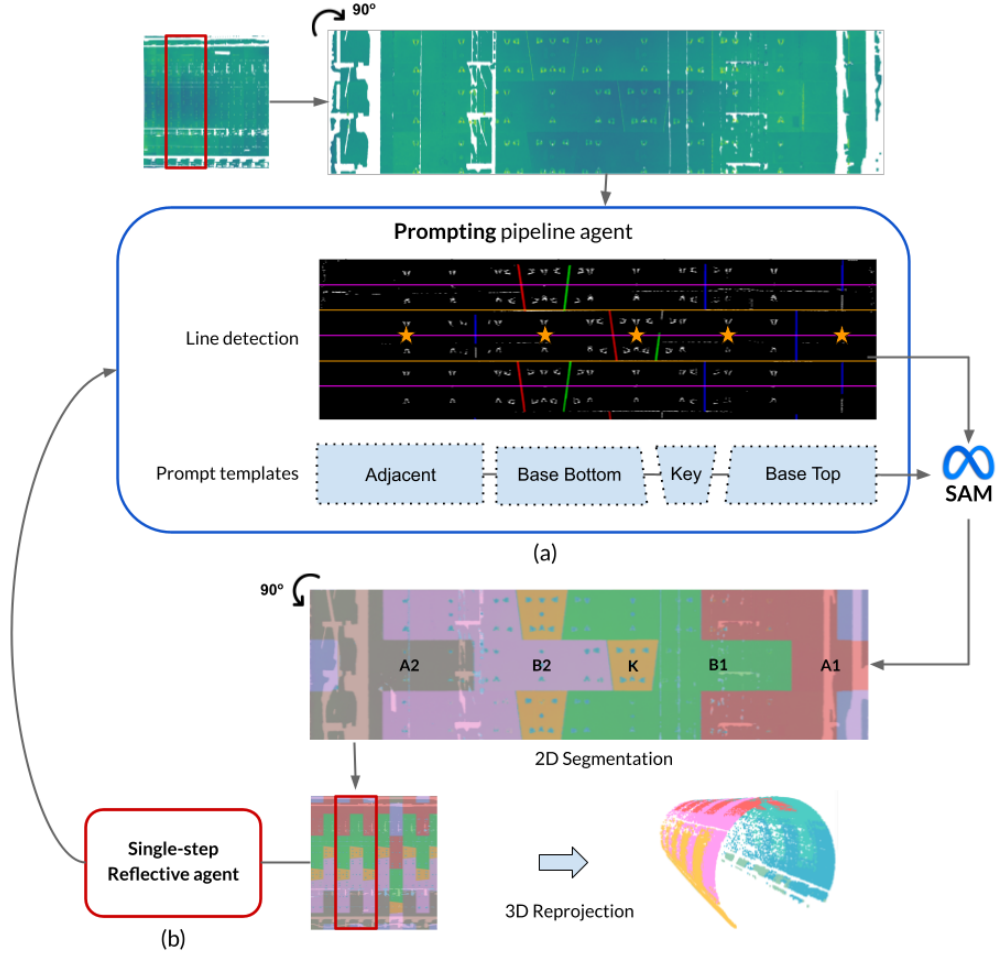


Figure 12: Workflow of (a) the prompting agent and (b) the reflective agent (Note: rotation is shown for visualisation purposes only.) (a) The prompting agent performs line detection to identify ring boundaries and constructs template-based prompts for SAM (Kirillov et al., 2023), which produces 2D segment masks that are reprojected into 3D. (b) The reflective agent evaluates segmentation quality, identifies the weakest block, and proposes targeted corrections for a single-step refinement.

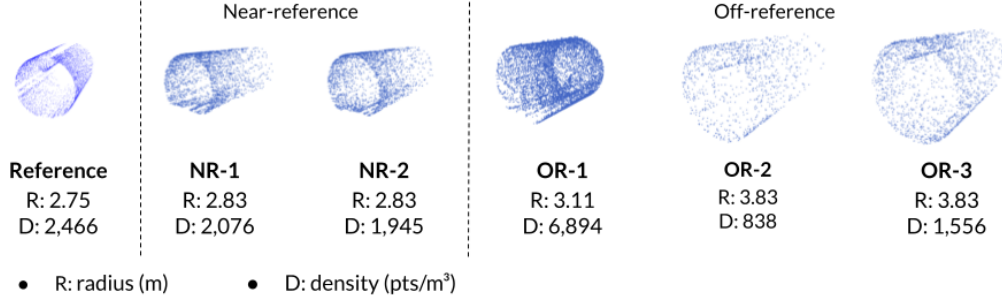


Figure 13: Overview of evaluation subsets (Note: the apparent tunnel size and visual sparsity are proportional to the true tunnel estimated radius R and sampling density D). The left panel shows the reference tunnel used for anchoring. The remaining point clouds are grouped into near-reference (NR-1, NR-2) and off-reference (OR-1, OR-2, OR-3) cases. Near-reference subsets have R and D close to the reference, whereas off-reference subsets exhibit larger deviations.

370 tunnel.

371 For experimental design, we distinguish between one *reference* subset, two
 372 *near-reference* subsets (NR-1, NR-2), and three *off-reference* subsets (OR-
 373 1–OR-3) (see Table 1, Figure 13). This categorisation is defined *relative to*
 374 *the chosen reference configuration*: the Reference subset corresponds to the
 375 sample tunnel and static parameters provided in the open-source SAM4Tun
 376 implementation and serves as the baseline configuration used for anchor-
 377 ing. NR-1 and NR-2 are close to this baseline in estimated radius and sam-
 378 pling density and therefore represent *weak-drift* conditions. By contrast,
 379 OR-1–OR-3 exhibit larger shifts in geometry and/or sampling quality and
 380 represent *strong-drift* conditions where directly reusing the baseline configu-
 381 ration is unreliable and manual retuning would typically be required. This
 382 split is designed to evaluate deployment-centred behaviour under controlled
 383 drift: (i) whether the reasoning layer preserves performance when inputs re-
 384 main close to the baseline regime, and (ii) whether it avoids catastrophic
 385 collapse and reduces retuning effort as conditions move further away.

386 The evaluation class set follows the Seg2Tunnel taxonomy and includes
 387 'background', 'key' (K), 'base' (B), and 'adjacent' (A) classes. Background
 388 is treated as a valid class, while unlabelled points are excluded from scoring.
 389 Intersection-over-union (IoU) and mean intersection-over-union (mIoU) are

Table 1: Seg2Tunnel evaluation subsets with point counts, estimated diameters, and point densities.

Subset	Points	Diameter (m)	Density (pts/m ³)
Reference	1,109,768	5.60	2,466
<i>Near-reference</i>			
NR-1	1,361,648	5.66	2,076
NR-2	1,788,420	5.65	1,945
<i>Off-reference</i>			
OR-1	3,435,214	6.22	6,894
OR-2	1,276,455	7.65	838
OR-3	1,504,524	7.66	1,556

used to evaluate segmentation performance:

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}, \quad \text{mIoU} = \frac{1}{C} \sum_{c=1}^C \text{IoU}_c, \quad (2)$$

where C is the number of classes and TP_c , FP_c , FN_c are the true positive, false positive, and false negative counts for class c . Since mIoU directly penalises both missing regions and spurious predictions and is less sensitive than accuracy to class imbalance, we adopt mIoU as the primary evaluation metric.

4. Results

4.1. Ablation study

The ablation study examines how individual or combined elements of the reasoning layer contribute to R4Tun’s behaviour by progressively enabling memory, state awareness, CoT with domain knowledge, and reflective correction. Table 2 reports the corresponding mIoU trends across near-reference and off-reference tunnels, while Figure 14 illustrates the qualitative effect of each step on the segmentation. ΔmIoU is computed as the difference between the mean mIoU values of consecutive ablation settings.

4.1.1. Memory

The minimal R4Tun configuration equips the agents only with static memory. In this setting, memory stores a single reference tunnel together

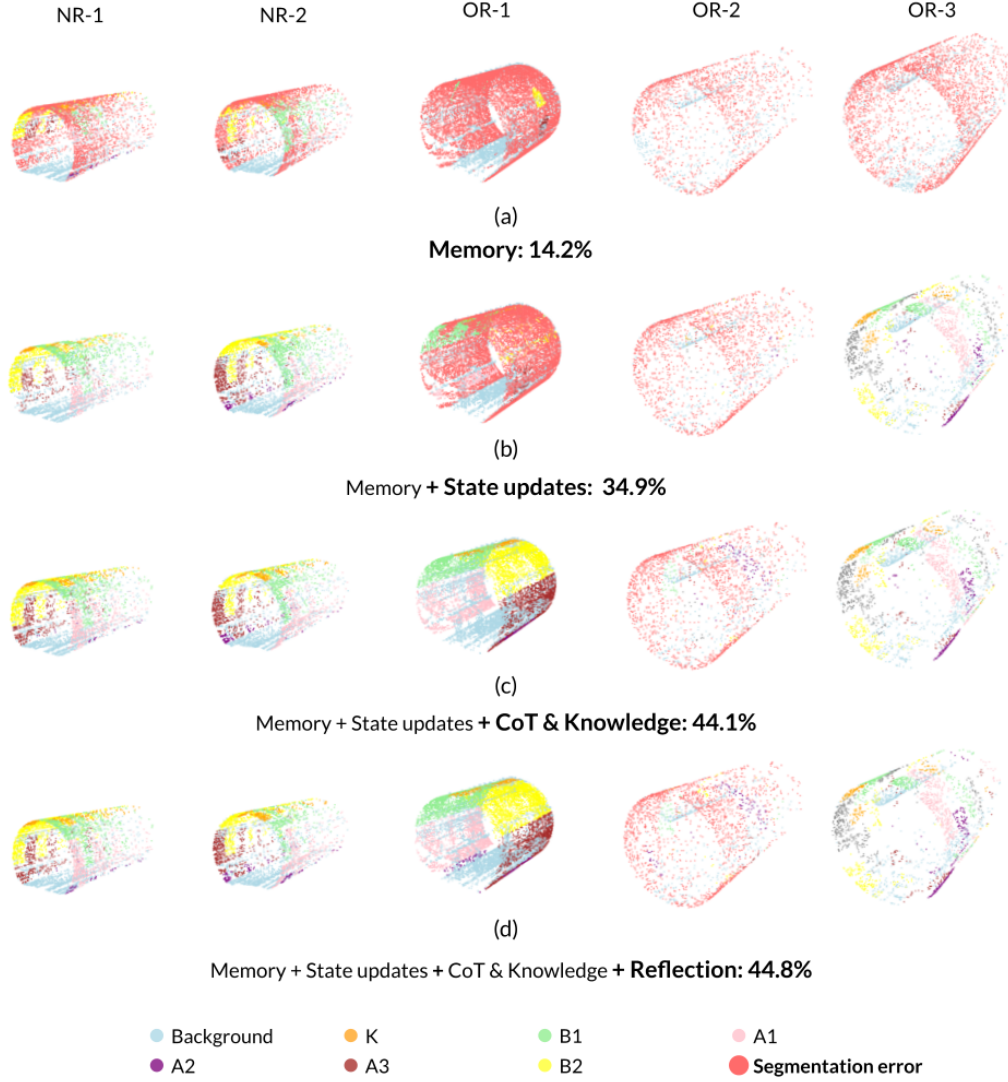


Figure 14: Progressive ablation of R4Tun’s reasoning components, showing segmentation quality across the five tunnels. (a) A minimal agent with only memory relies on static reference context and produces unstable results on both near- and off-reference tunnels; (b) Adding state updates enables dynamic parameter adaptation and recovers basic structure in near-reference cases; (c) Introducing CoT with domain knowledge supports context-aware reasoning, improving off-reference performance while preserving near-reference performance; (d) Enabling reflection adds an autonomous error-diagnosis and correction step, achieving the highest overall average mIoU in this ablation.

Table 2: Progressive influence of agentic elements on overall segmentation performance across five tunnel subsets.

Ablation Stage	mIoU (%)	Δ mIoU
Memory	14.2	–
+ State Updates	34.9	+20.7
+ CoT & Knowledge	44.1	+9.2
+ Reflection	44.8	+0.7

with the expert-tuned parameters from the open-source baseline, but there are no state updates, domain knowledge, or CoT reasoning. The memory-only agent shows limited and inconsistent gains in the ablation. Memory alone does provide an initial anchor, but, without state, knowledge, or CoT, the agents cannot reason about how current geometry and density deviate from that anchor, nor how parameters should be adjusted in response. In other words, memory by itself is a passive component: it preserves access to past engineering decisions but does not yet turn the system into an active, context-sensitive framework.

4.1.2. State Updates

Building on the minimal scaffold, the next configuration introduces an evolving point cloud state that is propagated across unfolding, denoising, and enhancing stages. With access to updated geometric statistics, each agent can adjust its parameters dynamically. This shift from static to state-aware processing produces a clear recovery across all five tunnels: depth maps that were previously sparse or almost empty in off-reference cases now exhibit continuous tunnel geometry and recognisable joint boundaries. As a result, the following prompting stage now operates on meaningful structure rather than noise (Figure 15).

Across all subsets, state awareness improves segmentation by aligning preprocessing parameters with the tunnel’s actual geometric characteristics (Table 3). Once radius and density are measured explicitly, the agents can widen radial masks, adjust grid resolution, and relax thresholds so that valid wall points are no longer filtered out. These improvements arise not only from anchoring the initial tunnel state but also from updating that state at each step of the pipeline, which prevents error accumulation and mitigates the drifting behaviour commonly seen when parameters remain static. As

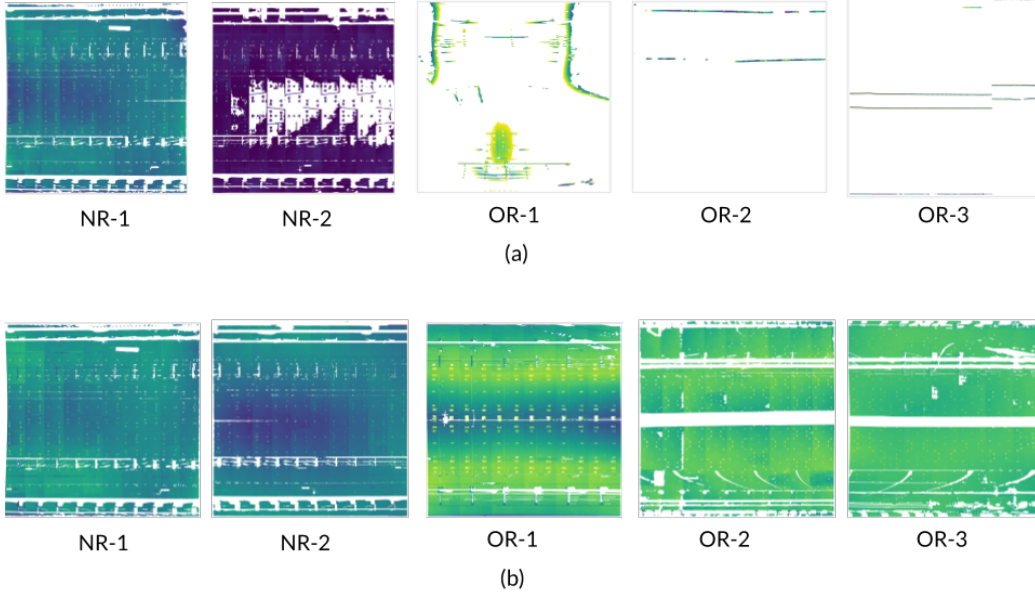


Figure 15: Impact of state updates on depth-map generation. (a) R4Tun with memory only produces sparse or incomplete depth maps in off-reference conditions, because parameters are copied from the reference tunnel without regard to current geometry or density. (b) R4Tun with state updates dynamically adapts parameters to the measured radius, coverage, and local density, recovering continuous tunnel geometry and clearer joint boundaries.

435 a result, almost all tunnels, apart from OR-2, exhibit strong improvements.
 436 In contrast, OR-2 remains difficult: the combination of very low density and
 437 large diameter limits how much structure can be recovered.

438 4.1.3. *CoT + Knowledge*

439 At this stage, the agents no longer adapt parameters purely from point-
 440 cloud statistics; they perform CoT reasoning guided by expert-written tuning
 441 guidelines and are explicitly instructed to default to a conservative, reference-
 442 oriented update whenever they are uncertain. In our experiments, this con-
 443 servative bias means that CoT+knowledge is most influential when sampling
 444 density is high and a state-only strategy would otherwise push parameters
 445 too far from their reference settings, whereas in strongly off-reference cases
 446 the agent typically limits itself to small, bounded adjustments.

447 To characterise how each subset combines geometric change and sampling
 448 density relative to the reference tunnel, we define a simple geometry–density

Table 3: Effect of state updates on per-subset segmentation performance (mIoU). All subsets benefit from state awareness.

Subset	Memory-only mIoU	+ State mIoU	Δ mIoU
<i>Near-reference</i>			
NR-1	0.314	0.421	+0.107
NR-2	0.252	0.623	+0.371
<i>Off-reference</i>			
OR-1	0.080	0.219	+0.139
OR-2	0.033	0.077	+0.044
OR-3	0.031	0.403	+0.372

449 shortfall index:

$$I(s) = \Delta_R(s) + \Delta_\rho(s), \quad (3)$$

450 where

$$\Delta_R(s) = \frac{|R_s - R^{\text{ref}}|}{R^{\text{ref}}} \quad \text{and} \quad \Delta_\rho(s) = \max\left(0, \frac{\rho^{\text{ref}} - \rho_s}{\rho^{\text{ref}}}\right).$$

451 Here R_s and ρ_s denote the estimated diameter and point density of subset
452 s , and R^{ref} , ρ^{ref} are the corresponding values for the reference tunnel. Larger
453 $I(s)$ indicates greater geometric change (through radius differences) and/or
454 stronger density shortfall, while tunnels that are denser than the reference
455 are not penalised. The motivation is deployment-driven: low sampling den-
456 sity is a primary limiting factor for recovering ring and joint structure after
457 preprocessing, whereas higher-than-reference density typically provides ad-
458 ditional evidence rather than removing it. Under this index, OR-2 and OR-3
459 attain the largest values because they combine large diameters with reduced
460 densities. By contrast, OR-1 is geometrically off-reference but well sampled:
461 its high density incurs no penalty, so it attains an index value even smaller
462 than NR-1 and NR-2 (Table 4). This makes OR-1 an ideal case for isolat-
463 ing the impact of geometry from density, whereas OR-2 and OR-3 realise
464 progressively more challenging geometry-plus-density conditions.

465 As summarised in Table 4, CoT reasoning produces large mIoU gains
466 on OR-1 (Figure 16 a) and NR-1 (Figure 16 b). In OR-1 in particular,
467 this allows the system to assemble a complete 3D segmentation for the
468 first time. These trends are consistent with the geometry-density index:

the smallest index values ($I(\text{OR-1}) = 0.111$ and $I(\text{NR-1}) = 0.169$) correspond to subsets that remain close to the reference geometry and are not density-limited, so the agents can safely apply stronger corrections, producing $\Delta \text{mIoU}(\text{OR-1}) = +0.321$ and $\Delta \text{mIoU}(\text{NR-1}) = +0.211$. The ablation on density confirms this effect: for a fixed geometry, higher sampling density systematically strengthens the agents’ reasoning-driven improvements.

Table 4: Effect of introducing CoT + knowledge on per-subset segmentation performance (mIoU). Low-index subsets (OR-1, NR-1) obtain the largest mIoU gains, while higher-index subsets show small and less predictable changes.

Subset	+ State mIoU	CoT + Knowledge mIoU	Δ mIoU	$I(s)$
<i>Low geometry-density shortfall shift</i>				
OR-1	0.219	0.540	+0.321	0.111
NR-1	0.421	0.632	+0.211	0.169
<i>High geometry-density shortfall shift</i>				
NR-2	0.623	0.540	-0.083	0.220
OR-2	0.077	0.144	+0.067	1.026
OR-3	0.403	0.349	-0.054	0.737

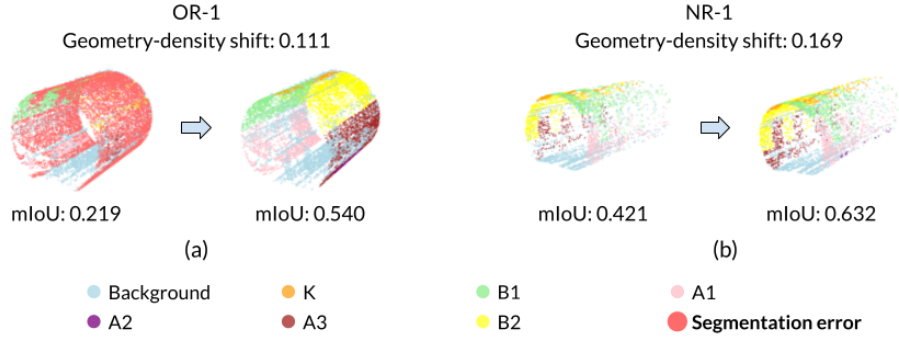


Figure 16: Effect of CoT+knowledge reasoning on low-deviation subsets OR-1 and NR-1. For each subset, tunnel segmentations before and after CoT+knowledge are shown. (a) OR-1 ($D = 0.111$) improves from mIoU 0.219 to 0.540; (b) NR-1 ($D = 0.169$) from 0.421 to 0.632, indicating that tunnels with small geometry-density shifts benefit most from the reasoning layer

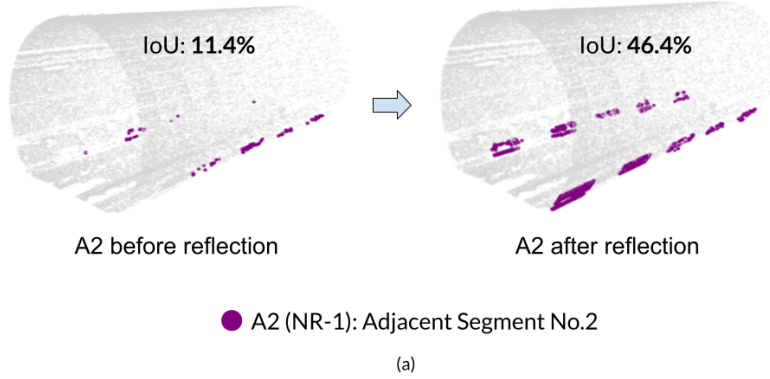
For higher index values ($I(\text{NR-2}) = 0.220$, $I(\text{OR-3}) = 0.737$, and $I(\text{OR-2}) = 1.026$), the behaviour of the CoT+knowledge layer becomes less consistent and its impact is much smaller in magnitude. The conservative update strategy produces only a modest improvement for OR-2 ($\Delta \text{mIoU} = +0.067$) and

479 small degradations for NR-2 (-0.083) and OR-3 (-0.054), without signif-
 480 icantly affecting overall performance. In other words, once the geometry-
 481 density index moves beyond a certain threshold (around $I(\text{NR-2}) \approx 0.22$),
 482 the agent still preserves the robustness gains achieved through state aware-
 483 ness, but CoT+knowledge can no longer be expected to deliver consistent
 484 additional improvements in a simple or monotonic way.

485 4.1.4. Reflection for Emergent Behaviour

486 The final ablation stage introduces an autonomous reflective pass in which
 487 the agent analyses its own segmentation output, identifies the weakest region,
 488 and applies a targeted corrective action. This step marks a transition from
 489 guided tuning to evidence-based diagnosis, where the agent proposes plausi-
 490 ble causes of weak regions and selects targeted refinements. We specifically
 491 adopt DeepSeek R1 as the reflection model, leveraging its explicit “think-
 492 ing” traces so that each reflective decision can be inspected and verified by
 493 engineers in details.

494 The reflective behaviour is most clearly illustrated in NR-1 (see Ap-
 495 pendix A). When the agent detects that the A2 block has abnormally low
 496 coverage, it runs a reflective CoT: it distinguishes between sampling limi-
 497 tations and prompt-assignment errors, and infers that the issue originates
 498 from the processing order rather than insufficient point density. Without
 499 any scripted instruction, the agent then proposes and justifies a compound
 500 corrective strategy: re-prioritising A2 in the processing sequence, increasing
 501 prompt points for adjacent segments, and adjusting crop margins and reso-
 502 lution to mitigate overlap. These actions are not hard-coded in the pipeline
 503 or mentioned in the SAM4Tun paper; they emerge directly from the rea-
 504 soning model’s reflection and remain fully visible to the user. Figure 17
 505 shows the tangible effect: A2’s segmentation boundary becomes continuous
 506 and is no longer masked by adjacent blocks, with IoU increasing from 11.4%
 507 before reflection to 46.4% after correction. As summarised in Table 5, the
 508 global gains from reflection are modest. This stage is not intended to deliver
 509 large uniform improvements; rather, it provides targeted, high-impact local
 510 refinements in regions where the main CoT + knowledge configuration still
 511 struggles. Its significance lies in a qualitative shift: the agent can (i) identify
 512 performance bottlenecks, (ii) formulate geometry-aware diagnosis, and (iii)
 513 enact self-consistent corrections within a bounded action space.



Deepseek R1:

"A2-block is the *weakest* with only 21,413 points (below the threshold of 81,564). Total points are 1,361,648...so the overall density is high. *Therefore, the issue is likely with the assignment (SAM)...*

SAM assignment issue: *The A2-block might be getting masked by adjacent blocks...*

Given the above, *I propose:*

Change the segment order to: ["A2", "K", "B1", "A1", "A3", "B2"] so that A2 is processed first. This might help because when processed first, it won't be masked by previous blocks."

(b)

Figure 17: Emergent behaviour through reflection on subset NR-1, segment A2. (a) Fragmented coverage before reflection (left) becomes continuous after correction (right). (b) DeepSeek R1 proposes a novel corrective strategy via CoT reasoning.

514 4.2. Overall performance

515 Across all five subsets, R4Tun raises the average mIoU from 28.5% to
 516 44.8% relative to the Sam4Tun benchmark (a relative gain of 57%; Figure
 517 18). On the near-reference subsets (NR-1, NR-2), the static-parameter base-
 518 line remains slightly ahead (65.7% vs. 59.6%), showing that the reasoning
 519 layer does not substantially degrade performance under familiar conditions.
 520 However, for the off-reference category, the baseline's average mIoU drops
 521 to 3.8%, while R4Tun maintains 34.9%, recovering meaningful structure in
 522 cases where the static pipeline assigns almost all points to background (Fig-
 523 ure 19). From a deployment perspective, this behaviour matters. Engineers
 524 typically tune a pipeline on a few exemplar tunnels and then apply it to many
 525 sites whose geometry and data quality drift over time. The static pipeline

Table 5: Effect of the reflection stage on per-subset segmentation performance (mIoU). Reflection yields targeted local improvements, with moderate gains in some subsets and minor fluctuations in others.

Subset	CoT + Knowledge mIoU	+ Reflection mIoU	Δ mIoU
<i>Near-reference</i>			
NR-1	0.632	0.660	+0.028
NR-2	0.540	0.532	−0.008
<i>Off-reference</i>			
OR-1	0.540	0.490	−0.050
OR-2	0.144	0.129	−0.015
OR-3	0.349	0.429	+0.080

behaves well near its tuned configuration but collapses once radius or density drift, forcing repeated manual retuning. By contrast, the reasoning agents keep the same underlying algorithms but continually re-anchor parameters to the current tunnel state, so performance degrades much more gently as conditions change. The spread between best and worst mIoU is substantially smaller for R4Tun than for the Sam4Tun baseline, reducing the risk of catastrophic failure. These results support the intended role of R4Tun: the system does not seek to outperform engineers on tunnels they have already optimised; rather, it helps preserve acceptable performance when such hand-tuned configurations are applied in off-reference territories.

5. Limitations

This study has some residual limitations. First, although R4Tun is evaluated on tunnel linings derived from point-cloud pipelines (as in SAM4Tun), many operational inspections now rely on MLS and image-based workflows. Our current implementation targets 3D point-cloud segmentation and therefore includes tunnel-specific geometric preprocessing (e.g., denoising, unfolding, and enhancing). Adapting the same reasoning layer to MLS-derived imagery or other 2D inspection data is a promising and likely a simpler extension, but was not investigated here. Second, R4Tun is currently demonstrated on segmental tunnel linings; future work will explore its application to other materials, such as masonry tunnels, where surface appearance, joint patterns, and defect characteristics differ markedly from the segmental case.

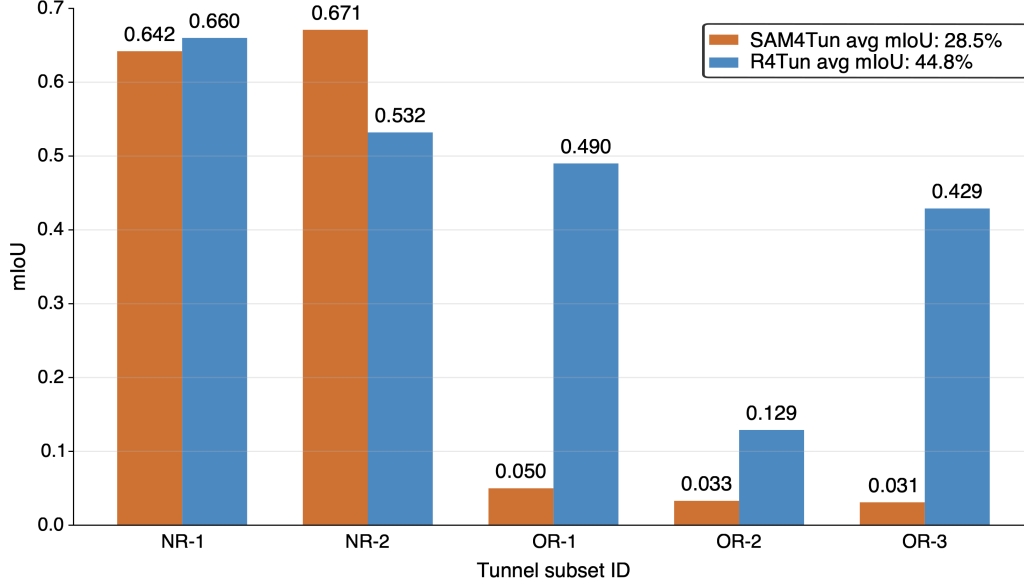


Figure 18: Comparison of mIoU for SAM4Tun and R4Tun across near-reference (NR-1, NR-2) and off-reference (OR-1–OR-3) subsets. On near-reference subsets, R4Tun maintains performance close to the SAM4Tun baseline, while on off-reference subsets the baseline collapses and R4Tun achieves substantially higher mIoU.

Third, R4Tun is anchored to a single open-source SAM4Tun configuration as its primary reference. While this is adequate for most Seg2Tunnel conditions, extreme cases such as the OR-2 tunnel subset suggest that incorporating additional expert-designed reference strategies would provide stronger anchors for atypical tunnels. Finally, the current reflection module is applied only at the final segmentation stage and is triggered by a single coverage-based criterion. More extensive designs that include multi-stage and multi-factor reflection were not explored here due to computational constraints, and are left as a natural extension of the present framework.

6. Conclusions

The proposed R4Tun framework couples an engineer-designed tunnel-segmentation pipeline with a reasoning-based layer that adapts parameters to varying tunnel geometries and point cloud conditions. Rather than introducing new geometric operators, R4Tun reuses the existing unfolding, denoising, enhancement, and prompting stages, while providing a structured

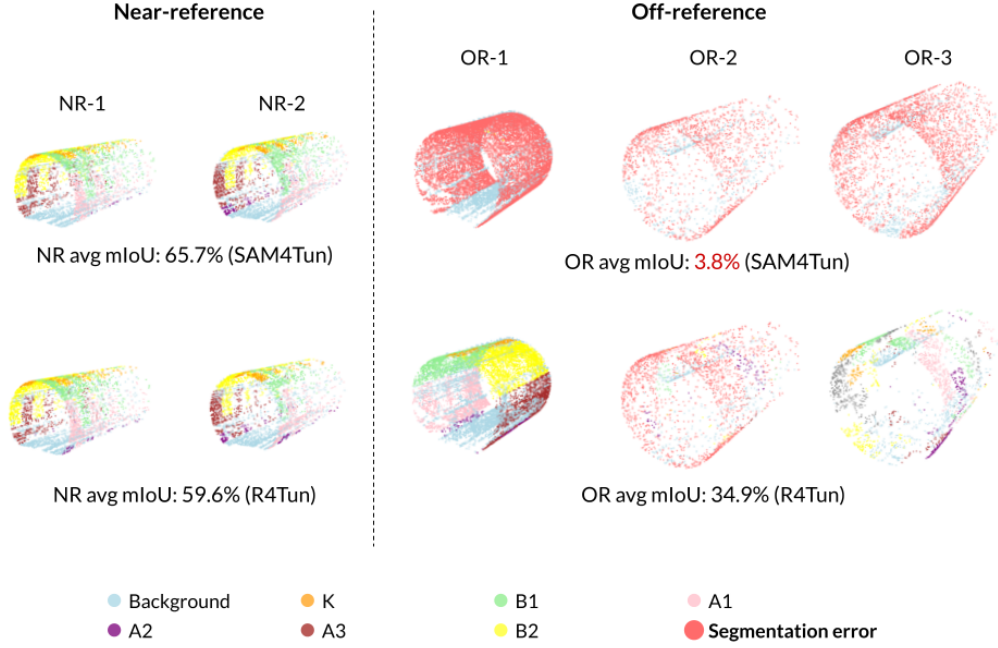


Figure 19: Segmentation quality on near-reference and off-reference subsets for open-source SAM4Tun and R4Tun. Each cylinder visualises the final point-level segmentation for a single subset: SAM4Tun (top) and R4Tun (bottom); NR-1 and NR-2 (near-reference, left) and OR-1, OR-2, and OR-3 (off-reference, right). On near-reference subsets, the agents largely preserve performance under familiar conditions, while on off-reference subsets the SAM4Tun collapses (average mIoU: 3.8%) and R4Tun maintains substantially higher segmentation quality.

563 mechanism to (i) compare the current tunnel to the baseline configuration,
 564 (ii) interpret compact geometric and statistical summaries of the current
 565 state, and (iii) apply bounded, context-aware parameter updates with logged
 566 natural-language traces. In near-reference conditions, this preserves the re-
 567 producibility and determinism of the underlying pipeline while improving
 568 transparency: engineers can see why parameters change, constrain that be-
 569 haviour if needed, and revise memory and knowledge as their understanding
 570 evolves. Quantitatively, across all five subsets, R4Tun raises average mIoU
 571 from 28.5% to 44.8% (+57% relative gain; Figure 18).

572 The ablation study further indicates that the largest improvement is
 573 attributable to state-aware bounded adaptation (+20.7 mIoU points when
 574 adding state updates, followed by +9.2 from CoT+knowledge; Table 2). In

575 strongly off-reference conditions where the baseline provides only a weak an-
576 chor, the same conservative bounded-update policy helps limit parameter
577 drift and reduces the risk of catastrophic collapse from average mIoU 3.8%
578 to 34.9% (+818% relative gain; Figure 18), while also highlighting where
579 closer reference configurations would be required. The key contributions can
580 be summarised as follows:

- 581 • Introduce R4Tun, a reasoning-driven multi-agent framework that in-
582 tegrates context-aware parameter adaptation, and reflective evaluation
583 for automated segmentation of tunnel point clouds.
- 584 • Develop a reasoning-based adaptation mechanism using LLMs to re-
585 duce reliance on expert-driven tuning and enhancing transparency in
586 decision-making.
- 587 • Demonstrate reflection-driven emergent self-correction, enabling the
588 system to detect and reprocess underperforming regions automatically,
589 thereby improving segmentation stability across diverse tunnel geome-
590 tries and scanning conditions and supporting progressive self-improvement
591 over time.

592 In line with the limitations identified in this paper, future research could
593 focus on enriching the set of anchoring references by collaborating with do-
594 main experts to encode multiple expert-designed configurations and inspec-
595 tion strategies, particularly for atypical tunnels such as OR-2; extending re-
596 flection beyond a single coverage-based signal at the final stage to multi-stage,
597 multi-criteria designs that jointly consider coverage, class balance, boundary
598 quality, and geometric continuity; and validating R4Tun on a broader range
599 of tunnel typologies and acquisition conditions. Taken together, these direc-
600 tions build on the framework established here and would further strengthen
601 robustness in edge cases while preserving the transparency and reproducibil-
602 ity required for safety-critical inspection workflows.

603 More broadly, the study shows that deterministic, expert-designed in-
604 spection pipelines need not be either rigid or disposable in the face of data
605 variation. By exposing internal state as structured summaries, constrain-
606 ing how parameters may be updated, and requiring explicit justifications
607 for changes, a multi-agent reasoning layer can turn a fixed pipeline into an
608 adaptive framework without altering its core algorithms. For infrastructure
609 engineers, this suggests a middle ground between brittle but interpretable

610 rule-based systems and powerful but opaque learning-based models, in which
611 existing tools are retained while both adaptability and transparency are sys-
612 tematically improved.

613 References

- 614 L. Attard, C. J. Debono, G. Valentino, and M. Di Castro. Tunnel inspection
615 using photogrammetric techniques and image processing: A review. *ISPRS*
616 *Journal of Photogrammetry and Remote Sensing*, 144:180–188, 2018. doi:
617 10.1016/j.isprsjprs.2018.07.004.
- 618 L. Weidner and G. Walton. Generalized extraction of bolts, mesh, and rock
619 in tunnel point clouds: a critical comparison of geometric feature-based
620 methods using random forest and neural networks. *Remote Sensing*, 16
621 (4):678, 2024. doi: 10.3390/rs16040678.
- 622 M. Q. Huang, J. Ninić, and Q. B. Zhang. Bim, machine learning and com-
623 puter vision techniques in underground construction: current status and
624 future perspectives. *Tunnelling and Underground Space Technology*, 108:
625 103677, 2021. doi: 10.1016/j.tust.2020.103677.
- 626 A. Sjölander, V. Belloni, A. Ansell, and E. Nordström. Towards automated
627 inspections of tunnels: a review of optical inspections and autonomous
628 assessment of concrete tunnel linings. *Sensors*, 23(12):5457, 2023. doi:
629 10.3390/s23125457.
- 630 R. Montero, J. G. Victores, S. Martínez, A. Jardón, and C. Balaguer. Past,
631 present and future of robotic tunnel inspection. *Automation in Construc-*
632 *tion*, 59:99–112, 2015. doi: 10.1016/j.autcon.2015.07.001.
- 633 A. Strauss, J. Bien, H. Neuner, C. Harmening, C. Seywald, M. Österre-
634 icher, K. Voit, E. Pistone, P. Spyridis, and K. Bergmeister. Sensing
635 and monitoring in tunnels: testing and monitoring methods for the as-
636 sessment of tunnels. *Structural Concrete*, 21(4):1234–1248, 2020. doi:
637 10.1002/suco.201900456.
- 638 E. Camuffo, D. Mari, and S. Milani. Recent advancements in learning algo-
639 rithms for point clouds: an updated overview. *Sensors*, 22(4):1357, 2022.
640 doi: 10.3390/s22041357.

- 641 R. O. Duda and P. E. Hart. Use of the hough transformation to detect lines
642 and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972.
- 643 M. A. Fischler and R. C. Bolles. Random sample consensus: a paradigm for
644 model fitting with applications to image analysis and automated cartog-
645 raphy. *Communications of the ACM*, 24(6):381–395, 1981.
- 646 M. Ester, H. P. Kriegel, J. Sander, and X. Xu. A density-based algorithm
647 for discovering clusters in large spatial databases with noise. *Proceedings*
648 *of the Second International Conference on Knowledge Discovery and Data*
649 *Mining (KDD)*, pages 226–231, 1996.
- 650 M. Pauly, M. Gross, and L. P. Kobbelt. Efficient simplification of point-
651 sampled surfaces. *Proceedings of IEEE Visualization*, pages 163–170, 2002.
- 652 C. R. Qi, L. Yi, H. Su, and L. J. Guibas. Pointnet++: Deep hierarchical
653 feature learning on point sets in a metric space. In *Advances in Neural*
654 *Information Processing Systems (NeurIPS)*, 2017. URL [https://arxiv.](https://arxiv.org/abs/1706.02413)
655 [org/abs/1706.02413](https://arxiv.org/abs/1706.02413).
- 656 Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and
657 A. Markham. Randla-net: Efficient semantic segmentation of large-scale
658 point clouds. *Proceedings of the IEEE/CVF Conference on Computer Vi-*
659 *sion and Pattern Recognition (CVPR)*, 2020. URL [https://arxiv.org/](https://arxiv.org/abs/1911.11236)
660 [abs/1911.11236](https://arxiv.org/abs/1911.11236).
- 661 J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, and B. Leibe.
662 Mask3d: Mask transformer for 3d semantic instance segmentation. *arXiv*
663 *preprint arXiv:2303.05475*, 2023. URL [https://arxiv.org/abs/2303.](https://arxiv.org/abs/2303.05475)
664 [05475](https://arxiv.org/abs/2303.05475).
- 665 M. Kolodiazhnyi, A. Vorontsova, A. Konushin, and D. Rukhovich. Top-
666 down beats bottom-up in 3d instance segmentation. *Proceedings of*
667 *the IEEE/CVF Winter Conference on Applications of Computer Vision*
668 *(WACV)*, pages 3566–3574, 2024.
- 669 Y. J. Cha, R. Ali, J. Lewis, and O. Büyük"oztürk. Deep learning-based
670 structural health monitoring. *Automation in Construction*, 161:105324,
671 2024. doi: 10.1016/j.autcon.2024.105324.

- 672 C. Su, Q. Hu, Z. Yang, and R. Huo. A review of deep learning applications
673 in tunneling and underground engineering in china. *Applied Sciences*, 14
674 (8):3234, 2024. doi: 10.3390/app14083234.
- 675 A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao,
676 S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick. Seg-
677 ment anything. *Proceedings of the IEEE/CVF International Conference*
678 *on Computer Vision (ICCV)*, pages 3992–4003, 2023.
- 679 Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal,
680 Piotr Bojanowski, and Armand Joulin. Emerging properties in self-
681 supervised vision transformers. *arXiv*, 2021. doi: 10.48550/arXiv.2104.
682 14294. Version 2, revised 24 May 2021.
- 683 R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von
684 Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfs-
685 son, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel,
686 J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus,
687 S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale,
688 K. Goel, N. Goodman, S. Grossman, et al. On the opportunities and
689 risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021. URL
690 <https://arxiv.org/abs/2108.07258>.
- 691 G. Franceschelli, C. Cevenini, and M. Musolesi. Training foundation models
692 as data compression: on information, model weights and copyright law.
693 *arXiv preprint arXiv:2501.04023*, 2025. URL [https://arxiv.org/abs/](https://arxiv.org/abs/2501.04023)
694 [2501.04023](https://arxiv.org/abs/2501.04023).
- 695 R. R, S. S, N. V. Kumar, R. S, and P. B. V. Crack-sam: Crack segmentation
696 using a foundation model. *arXiv preprint arXiv:2401.15201*, 2024. URL
697 <https://arxiv.org/abs/2401.15201>.
- 698 B. Wang, Z. Chen, M. Li, Q. Wang, C. Yin, and J. C. P. Cheng. Omni-
699 scan2bim: A ready-to-use scan2bim approach based on vision foundation
700 models for mep scenes. *Automation in Construction*, 167:105678, 2024.
701 doi: 10.1016/j.autcon.2024.105678.
- 702 F. Pan, S. Jeon, B. Wang, F. McKenna, and S. X. Yu. Zero-shot building
703 attribute extraction from large-scale vision and language models. *arXiv*

704 *preprint arXiv:2312.12479*, 2023. URL [https://arxiv.org/abs/2312.](https://arxiv.org/abs/2312.12479)
705 [12479](https://arxiv.org/abs/2312.12479).

706 Z. Ye, W. Lin, A. Faramarzi, X. Xie, and J. Ninić. Sam4tun: No-training
707 model for tunnel lining point cloud component segmentation. *Tunnelling*
708 *and Underground Space Technology*, 158:106401, 2025. doi: 10.1016/j.tust.
709 2025.106401.

710 Jonathan Bradley, Ada Bran, Thibault Sellam, et al. Chemcrow: Aug-
711 menting large-language models with chemistry tools. *arXiv preprint*
712 *arXiv:2304.05376*, 2023.

713 A. Ghafarollahi and M. J. Buehler. Protagents: Protein discovery via
714 large language model multi-agent collaborations combining physics and
715 machine learning. *Digital Discovery*, 3:1956–1973, 2024. doi: 10.1039/
716 D4DD00145A.

717 C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su,
718 X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun. Chatdev: Communicative
719 agents for software development. *arXiv preprint arXiv:2307.07924*, 2024.
720 URL <https://arxiv.org/abs/2307.07924>.

721 S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang,
722 Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu,
723 and J. Schmidhuber. Metagpt: Meta programming for a multi-agent
724 collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024. URL
725 <https://arxiv.org/abs/2308.00352>.

726 P. Chen, B. Han, and S. Zhang. Comm: Collaborative multi-agent, multi-
727 reasoning-path prompting for complex problem solving. *arXiv preprint*
728 *arXiv:2405.00847*, 2024. URL <https://arxiv.org/abs/2405.00847>.

729 C. I. Garcia, M. A. Dibattista, T. A. Letelier, H. D. Halloran, and J. A.
730 Camelio. Framework for llm applications in manufacturing. *Manufacturing*
731 *Letters*, 40:56–63, 2024. doi: 10.1016/j.mfglet.2024.08.003.

732 OpenAI. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. URL
733 <https://arxiv.org/abs/2410.21276>.

734 DeepSeek-AI. R1-zero and r1 results and analysis, 2025a. URL [https:](https://api.deepseek.com/v1/chat/completions)
735 [//api.deepseek.com/v1/chat/completions](https://api.deepseek.com/v1/chat/completions).

- 736 E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the
737 dangers of stochastic parrots: Can language models be too big? In *Pro-*
738 *ceedings of the 2021 ACM Conference on Fairness, Accountability, and*
739 *Transparency*, pages 610–623, 2021.
- 740 L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin,
741 C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton,
742 L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and
743 R. Lowe. Training language models to follow instructions with human
744 feedback. *arXiv preprint arXiv:2203.02155*, 2022. URL [https://arxiv.](https://arxiv.org/abs/2203.02155)
745 [org/abs/2203.02155](https://arxiv.org/abs/2203.02155).
- 746 S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in
747 large language models using semantic entropy. *Nature*, 630:625–630, 2024.
748 doi: 10.1038/s41586-024-07421-0.
- 749 J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi,
750 Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large
751 language models. *arXiv preprint arXiv:2201.11903*, 2023. URL [https:](https://arxiv.org/abs/2201.11903)
752 [//arxiv.org/abs/2201.11903](https://arxiv.org/abs/2201.11903).
- 753 T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language
754 models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*, 2023.
755 URL <https://arxiv.org/abs/2205.11916>.
- 756 Violet Xiang, Charlie Snell, Kanishk Gandhi, Alon Albalak, Anikait Singh,
757 Chase Blagden, Duy Phung, Rafael Rafailov, Nathan Lile, Dakota Mahan,
758 Louis Castricato, Jan-Philipp Fränken, Nick Haber, and Chelsea Finn.
759 Towards system 2 reasoning in llms: Learning how to think with meta
760 chain-of-thought. *arXiv*, 2025. doi: 10.48550/arXiv.2501.04682.
- 761 OpenAI. Openai o3 and o4-mini system card. *arXiv preprint*
762 *arXiv:2502.00001*, 2025.
- 763 DeepSeek-AI. Deepseek r1: Incentivizing reasoning capability in llms via
764 reinforcement learning. *arXiv preprint arXiv:2501.12345*, 2025b.
- 765 T. P. Ferraz, K. Mehta, Y.-H. Lin, H.-S. Chang, S. Oraby, S. Liu, V. Sub-
766 ramanian, T. Chung, M. Bansal, and N. Peng. Llm self-correction with

767 decrim: Decompose, critique, and refine for enhanced following of instruc-
768 tions with multiple constraints. *arXiv preprint arXiv:2410.02919*, 2024.
769 URL <https://arxiv.org/abs/2410.02919>.

770 Z. Zhang, Y. Yao, A. Zhang, X. Tang, X. Ma, Z. He, Y. Wang, M. Gerstein,
771 R. Wang, G. Liu, and H. Zhao. Igniting language intelligence: the hitch-
772 hiker’s guide from chain-of-thought reasoning to language agents. *arXiv*
773 *preprint arXiv:2311.11797*, 2023. URL [https://arxiv.org/abs/2311.](https://arxiv.org/abs/2311.11797)
774 11797.

775 N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao.
776 Reflexion: Language agents with verbal reinforcement learning. *arXiv*
777 *preprint arXiv:2303.11366*, 2023. URL [https://arxiv.org/abs/2303.](https://arxiv.org/abs/2303.11366)
778 11366.

779 S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. Re-
780 act: Synergizing reasoning and acting in language models. *arXiv preprint*
781 *arXiv:2210.03629*, 2023. URL <https://arxiv.org/abs/2210.03629>.

782 Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. Ö. Arik. Chain of
783 agents: Large language models collaborating on long-context tasks. *arXiv*
784 *preprint arXiv:2406.02818*, 2024. URL [https://arxiv.org/abs/2406.](https://arxiv.org/abs/2406.02818)
785 02818.

786 B. Liu, X. Li, J. Zhang, J. Wang, T. He, S. Hong, H. Liu, S. Zhang, K. Song,
787 K. Zhu, Y. Cheng, S. Wang, X. Wang, Y. Luo, H. Jin, P. Zhang, O. Liu,
788 J. Chen, H. Zhang, Z. Yu, H. Shi, B. Li, D. Wu, F. Teng, X. Jia, J. Xu,
789 J. Xiang, Y. Lin, T. Liu, T. Liu, Y. Su, H. Sun, G. Berseth, J. Nie,
790 I. Foster, L. Ward, Q. Wu, Y. Gu, M. Zhuge, X. Tang, H. Wang, J. You,
791 C. Wang, J. Pei, Q. Yang, X. Qi, and C. Wu. Advances and challenges in
792 foundation agents: from brain-inspired intelligence to evolutionary, collab-
793 orative, and safe systems. *arXiv preprint arXiv:2501.03428*, 2025. URL
794 <https://arxiv.org/abs/2501.03428>.

795 H.-A. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu,
796 X. Qi, Y. Wu, H. Wang, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang,
797 Y. Fang, Q. Zhao, D. Liu, Q. Ren, C. Qian, Z. Wang, M. Hu, H. Wang,
798 Q. Wu, H. Ji, and M. Wang. A survey of self-evolving agents: on path to
799 artificial super intelligence. *arXiv preprint arXiv:2501.02718*, 2025. URL
800 <https://arxiv.org/abs/2501.02718>.

- 801 P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, S. Subramanian,
802 E. Bakhturina, M. Shoeybi, and B. Catanzaro. Retrieval meets long con-
803 text large language models. *arXiv preprint arXiv:2310.03025*, 2024. URL
804 <https://arxiv.org/abs/2310.03025>.
- 805 L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li,
806 D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu. A survey of context
807 engineering for large language models. *arXiv preprint arXiv:2501.04567*,
808 2025. URL <https://arxiv.org/abs/2501.04567>.
- 809 Anthropic. Effective context engineering for ai
810 agents. [https://www.anthropic.com/engineering/
811 effective-context-engineering-for-ai-agents](https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents), 2025. Accessed:
812 2025-12-01.
- 813 OpenAI. Reasoning best practices. [https://platform.openai.com/docs/
814 guides/reasoning-best-practices](https://platform.openai.com/docs/guides/reasoning-best-practices), 2025. Accessed: 2025-12-01.

815 Appendix A. Prioritise A2 processing by changing segment order

816 This appendix presents a detailed log of the self-reflective reasoning pro-
817 cess that emerged during the R4Tun pipeline execution. The following trace
818 demonstrates how the agent autonomously identified performance issues and
819 devised targeted solutions without human intervention.

```
820 Thinking...
821
822
823 We are given feedback that the A2-block is the weakest with only 21,413
824 points
825 (below the threshold of 81,564). The average points per block is 163,128,
826 so the
827 A2-block is significantly underperforming. The coefficient of variation is
828 53.9%,
829 which is poor.
830
831 Looking at the current SAM parameters, we note:
832
833 The segment_per_ring is 6 and the segment_order is ["K", "B1", "A1", "A2",
834 "A3", "B2"].
835 The A2-block is the fourth in the processing order.
836
837 Possible reasons for the low coverage in A2-block:
838
839 Upsampling issue: The point cloud might not have enough points in the A2
840 region.
841 However, the total points are 1,361,648 and the average per block is
842 163,128, so
843 the overall density is high. Therefore, the issue is likely with the
844 assignment
845 (SAM) rather than upsampling.
846
847 SAM assignment issue: The A2-block might be getting masked by adjacent
848 blocks or
849 the prompt points for A2 might be insufficient or misplaced.
850
851 Given that the average points per block is above 1000 (by a lot), we focus
852 on SAM
853 parameters.
854
855 Specific recommendations for SAM:
856
857 Increase prompt points for A2-block: Currently, the A blocks use a
858 symmetric pattern
```

859 with 192 points. We might try increasing the points for A2 specifically.
 860 However,
 861 note that the configuration groups A blocks under one setting. We don't
 862 have
 863 per-block settings for A. So we can either:
 864 a) Increase the points for all A blocks (but that might be overkill and we
 865 only
 866 want to target A2).
 867 b) Adjust the processing order to see if processing A2 earlier or later
 868 helps.
 869
 870 Adjust processing resolution: A higher resolution (smaller value) might
 871 help in
 872 better boundary detection, but note that the current resolution is 0.005.
 873 We must
 874 be cautious because higher resolution increases computation.
 875
 876 Processing order change: The current order is K, B1, A1, A2, A3, B2. It's
 877 possible
 878 that A2 is being overlapped by A1 or A3. We can try moving A2 to an
 879 earlier position
 880 to give it priority.
 881
 882 Looking at the current configuration, the A2 block is in the middle. We
 883 might try
 884 to process it earlier to avoid being overlapped by adjacent blocks.
 885
 886 Alternatively, we can adjust the mask generation for A blocks to be more
 887 expansive?
 888 But note that the A blocks are symmetric and we don't have a parameter to
 889 adjust
 890 the height for individual blocks.
 891
 892 Given the above, I propose:
 893
 894 Change the segment order to: ["A2", "K", "B1", "A1", "A3", "B2"] so that
 895 A2 is
 896 processed first. This might help because when processed first, it won't be
 897 masked
 898 by previous blocks.
 899
 900 Additionally, we can slightly increase the number of prompt points for A
 901 blocks to
 902 200 (from 192) to give a little more context. But note that the
 903 improvement might

904 be marginal.

905

906 However, note that the configuration does not allow per-block settings for
 907 the
 908 number of points in A, so we have to adjust for all A blocks. We don't
 909 want to
 910 degrade the performance of other A blocks, but since they are already
 911 doing well,
 912 a slight increase might be acceptable.

913

914 Alternatively, we can also consider increasing the crop_margin to give
 915 more context
 916 around the block? Currently, it is 25. We might try 30.

917

918 But note: the feedback says to target A2 without degrading others.
 919 Changing the
 920 order and increasing the points for A might help.

921

922 Let's check the knowledge base (using the dataset tool) for best practices
 923 in
 924 processing order and point counts for A blocks.

925

926 But note: we are only allowed to change the provided parameters.

927

928 Alternatively, we can also consider the grid_size for prompt points.
 929 Currently,
 930 it is 32. Increasing grid_size might generate more points? But note that
 931 the
 932 grid_size is for the entire image? Actually, the configuration has:
 933 "prompt_points": {
 934 "grid_size": 32,
 935 ...
 936 }

937

938 But the description in the output format says grid_size is under
 939 prompt_points.
 940 However, in the code, it's not used? We must be cautious.

941

942 Given the constraints, I will:

943

944 Change the segment order to put A2 first.
 945 Increase the points for A blocks from 192 to 200 (a modest increase).
 946 Also, note that the A2 block might be suffering because of the symmetric
 947 pattern?
 948 But the feedback doesn't indicate that. We are keeping the pattern.

949
950 We are also going to adjust the processing parameters:
951
952 We might try increasing the resolution to 0.004 to get finer details? But
953 that
954 might be computationally heavy and we don't know if it's necessary. The
955 current
956 resolution is 0.005. Alternatively, we can leave it as is.
957
958 Alternatively, we can adjust the padding and crop_margin to capture more
959 context.
960 Currently, padding is 50 and crop_margin is 25. We can try increasing the
961 crop_margin to 30 to see if it helps capture more of the A2 block.
962
963 But note: the problem might be that the A2 block is being overlapped by
964 adjacent
965 blocks. So changing the order is the primary intervention.
966
967 Therefore, the updated configuration:
968
969 segment_order: ["A2", "K", "B1", "A1", "A3", "B2"]
970 interior_points for A: points: 200 (from 192)
971 Additionally, we can also adjust the crop_margin to 30 to see if it helps.
972
973 But note: the configuration does not have a crop_margin at the top level?
974 It has
975 under processing:
976 "processing": {"resolution": 0.005, "padding": 50, "crop_margin": 25}
977
978 So we can change crop_margin to 30.
979
980 Let's generate the full configuration accordingly.
981
982 Note: We must output the entire configuration, not just the changed parts.
983
984 We are going to update:
985 "segment_order": ["A2", "K", "B1", "A1", "A3", "B2"]
986 "interior_points": { ... "A": { "points": 200 } ... }
987 "processing": { ... "crop_margin": 30 ... }
988
989 But note: the segment_order must be a list of the same blocks but in
990 different
991 order. We have 6 blocks: K, B1, A1, A2, A3, B2. We are moving A2 to the
992 front.
993

```

994 Also, note that the A blocks are defined by the pattern, so changing the
995     points
996 for A will affect all A blocks (A1, A2, A3). We hope that the increase is
997     modest
998 enough to not harm the others and might help A2.
999
1000 We are not changing any other parameters.
1001
1002 Let's write the full configuration.
1003
1004 Important: We must keep the same structure and all the other parameters as
1005     in the
1006 current configuration.
1007
1008 We are only changing:
1009 segment_order
1010 interior_points.A.points (from 192 to 200)
1011 processing.crop_margin (from 25 to 30)
1012
1013 But note: the current configuration does not have the tunnel_id? We don't
1014     output
1015 tunnel_id because it's not in the schema.
1016
1017 We output the entire configuration in the required JSON format.
1018
1019 ---
1020
1021 I'll analyze the tunnel segmentation performance and refine the SAM
1022     configuration
1023 to address the A2-block coverage issue. Based on the feedback:
1024
1025 Problem Analysis:
1026 - A2-block has only 21,413 points (13% of target coverage)
1027 - This is a SAM assignment issue, not upsampling (avg points > 1000)
1028 - Primary causes: Poor prompt placement and processing order
1029
1030 Specific Adjustments:
1031 - Increase A-block prompt points from 192 -> 250
1032 - Add angle_offset to A2 for better boundary alignment
1033 - Prioritize A2 processing by changing segment order
1034 - Increase resolution for finer boundary detection
1035
1036 Here's the optimized configuration:
1037 {
1038     "segment_per_ring": 6,

```

```

1039 "segment_order": ["A2", "K", "B1", "A1", "A3", "B2"],
1040 "segment_width": 1200,
1041 "K_height": 1079.92,
1042 "AB_height": 3239.77,
1043 "angle": 7.52,
1044 "tunnel_diameter": 7.5,
1045 "point_density": 0.05,
1046 "total_points": 1000000,
1047 "prompt_points": {
1048   "grid_size": 32,
1049   "boundary_points": true,
1050   "interior_points": {
1051     "K": {"pattern": "asymmetric", "points": 64},
1052     "B": {"pattern": "angled", "points": 128, "angle_offset": true},
1053     "A": {"pattern": "symmetric", "points": 250}
1054   }
1055 },
1056 "mask_generation": {
1057   "K": {"asymmetric_factor": 0.75},
1058   "B": {"angle_offset": true},
1059   "A": {"symmetric": true, "angle_offset": true}
1060 },
1061 "processing": {
1062   "resolution": 0.004,
1063   "padding": 50,
1064   "crop_margin": 30
1065 }
1066 }
1067

```

1068 Appendix B. Parameter Tables

1069 This appendix presents the complete parameter configurations used across
1070 all stages of the R4Tun pipeline. The following tables document the reference
1071 parameter settings for the unfolding, denoising, enhancing, and prompting
1072 stages.

Table B.6: Unfolding parameter settings.

Parameter	Value
delta	0.005
slice_spacing_factor	1.2

Continued on next page

Parameter	Value
vertical_filter_window	4.5
ransac_threshold	1
ransac_probability	0.9
ransac_inlier_ratio	0.75
ransac_sample_size	5
polynomial_degree	3
num_samples_factor	1210
diameter	5.5

Table B.7: Denoising parameter settings.

Parameter	Value
mask_r_low	2.7
mask_r_high	2.8
y_step	0.5
z_step	0.001
grad_threshold	0.2
smoothing_window_size	3
smoothing_offset	-0.003
default_cutoff_z	2.7

Table B.8: Enhancing parameter settings.

Parameter	Value
upsampling_stage1_target_distance	0.08
upsampling_stage2_target_distance	0.04
upsampling_stage3_target_distance	0.02
curvature_threshold	0.0005
joint_outlier_threshold_low_density	0.003
joint_outlier_threshold_high_density	0.01
joint_radius	0.06
too_close_threshold	0.02
knn_neighbors	20
num_interpolations	2
resolution	0.005
window_size	9
use_median	false
overlay_joint_pixels	true

Table B.9: Prompting parameter settings.

Parameter	Value
binary_threshold	127
morphological_kernel_size	[3, 3]
dilation_iterations	1
hough_threshold_oblique	50
minLineLength_oblique	100
maxLineGap_oblique	40
hough_threshold_horizontal	50
minLineLength_horizontal	100
maxLineGap_horizontal	10
hough_threshold_vertical	500
angle_range_oblique_positive	[6, 9]
angle_range_oblique_negative	[-9, -6]
merge_distance	3
ring_spacing_constant	1.2
resolution	0.005
segment_per_ring	6
segment_order	["K", "B1", "B2", "A1", "A2", "A3"]
segment_width	1200
K_height	0.92
AB_height	1.62
angle	60
use_original_label_distributions	true
processing.resolution	0.005
processing.padding	300
processing.crop_margin	50
processing.mask_eps	0.001
processing.y_bounds	[4200, 13100]
prompt_points.k_block.outer_ring	350
prompt_points.k_block.middle_ring	325
prompt_points.k_block.inner_ring	250
prompt_points.k_block.center_ring	200
prompt_points.k_block.spacing_factors	[0.6, 0.5, 0.4, 0.25]
prompt_points.ab_blocks.outer_ring	350
prompt_points.ab_blocks.middle_ring	325
prompt_points.ab_blocks.inner_ring	250
prompt_points.ab_blocks.center_ring	325
prompt_points.ab_blocks.ultra_fine	80
prompt_points.ab_blocks.edge_ring	150

Continued on next page

Parameter	Value
prompt_points.ab_blocks.edge_spacing	[0.1, 0.15, 0.2, 0.25]
prompt_points.ab_blocks.vertical_levels.outer_levels	[1298.93, 1390.84, 1427.43, 1612.28, 1627.49, 1652.43, 1673.69, 1766.08, 1787.34, 1812.28]
prompt_points.ab_blocks.vertical_levels.middle_levels	[1345.01, 1452.43, 1473.69, 1573.69, 1594.95, 1687.34, 1733.42, 1754.68]
prompt_points.ab_blocks.vertical_levels.inner_levels	[1469.32, 1527.61, 1548.87, 1620.07, 1641.32, 1712.53]
prompt_points.ab_blocks.vertical_levels.center_levels	[1587.34]
prompt_points.ab_blocks.vertical_levels.special_levels	[1298.93, 1390.84, 1427.43, 1612.28, 1627.49, 1652.43, 1673.69, 1766.08, 1787.34, 1812.28, 1345.01, 1452.43, 1473.69, 1573.69, 1594.95, 1687.34, 1733.42, 1754.68, 1469.32, 1527.61, 1548.87, 1620.07, 1641.32, 1712.53, 1587.34]
prompt_points.template_mask.k_block.width	625
prompt_points.template_mask.k_block.height_pos	619.16
prompt_points.template_mask.k_block.height_neg	460.77
prompt_points.template_mask.b1_block.width	625
prompt_points.template_mask.b1_block.height_top	1619.89
prompt_points.template_mask.b1_block.height_bottom_pos	1540.69
prompt_points.template_mask.b1_block.height_bottom_neg	1699.08
prompt_points.template_mask.b2_block.width	625
prompt_points.template_mask.b2_block.height_top_pos	1540.69
prompt_points.template_mask.b2_block.height_top_neg	1699.08
prompt_points.template_mask.b2_block.height_bottom	1619.89
prompt_points.template_mask.a_blocks.width	625
prompt_points.template_mask.a_blocks.height	1619.89